

An
PROJECT REPORT
ON
AUTOMATED GENERATION AND EVALUATION OF MCQ'S USING LANGCHAIN & GEMINI LLM

Submitted in partial fulfilment of the requirements for the degree of
BACHELOR OF TECHNOLOGY
in
ARTIFICIAL INTELLIGENCE AND DATA SCIENCE

Submitted by

T.DILEEP	22AK1A3022
D.LAKSHITHA	22AK1A3042
B.AKHILA	22AK1A3005
L.GAYATHRI	22AK1A3027

Under guidance of

Ms. A. Sathvika, MCA
Assistant Professor, AI&DS Department



DEPARTMENT OF ARTIFICIAL INTELLIGENCE AND DATA SCIENCE
ANNAMACHARYA INSTITUTE OF TECHNOLOGY AND SCIENCES, TIRUPATI
(AUTONOMOUS)

Approved by AICTE, New Delhi & Permanent Affiliation to JNTUA, Anantapuram. Five B.Tech Programmes (ECE, CSE, EEE, CE, ME) are accredited by NBA, New Delhi. Accredited by NAAC with 'A' Grade, Bangalore.
Accredited by Institution of Engineers (India) KOLKATA
A-grade awarded by AP Knowledge Mission. Recognized under sections 2(f)&12(B) of UGC Act 1956,
Venkatapuram (V), Renigunta (M), Tirupati, Tirupati District, Andhra Pradesh-517520
2025-2026

**ANNAMACHARYA INSTITUTE OF TECHNOLOGY AND SCIENCES, TIRUPATI
(AUTONOMOUS)**

DEPARTMENT OF ARTIFICIAL INTELLIGENCE AND DATA SCIENCE



CERTIFICATE

Certified that this is a bonafide record of the project report entitled, **“AUTOMATED GENERATION AND EVALUATION OF MCQ’s USING LANGCHAIN & GEMINI LLM** carried out by **T DILEEP (22AK1A3022), D LAKSHITHA (22AK1A3042), B AKHILA (22AK1A3005), L GAYATHRI (22AK1A3027)**. This project is submitted in partial fulfilment of the requirements for the award of the degree of **BACHELOR OF TECHNOLOGY** in **ARTIFICIAL INTELLIGENCE AND DATA SCIENCE** at Annamacharya Institute of Technology and Sciences, Tirupati (Autonomous), during the academic year 2025 – 2026.

Signature of the Guide

Ms. A. Sathvika, MCA
Asst. Professor
Department of AI & DS
AITS, Tirupati

Signature of Head of Department

Mrs. B. Rupa Devi Singh, M. Tech (Ph.D)
Associate Professor
Department of AI & DS
AITS, Tirupati

Internal Examiner

External Examiner

ANNAMACHARYA INSTITUTE OF TECHNOLOGY AND SCIENCES, TIRUPATI
(AUTONOMOUS)

DEPARTMENT OF ARTIFICIAL INTELLIGENCE & DATA SCIENCE



DECLARATION

We hereby declare that the project titled “**AUTOMATED GENERATION AND EVALUATION OF MCQ’S USING LANGCHAIN & GEMINI LLM** ” a genuine project work carried out by us a part of the B.Tech in Artificial Intelligence and Data Science at Annamacharya Institute of Technology and Sciences, Tirupati and has not been submitted to any other university or institution for the award of any degree.

T.DILEEP

22AK1A3022

D.LAKSHITHA

22AK1A3042

B.AKHILA

22AK1A3005

L.GAYATHRI

22AK1A3027

Place:

Signature

Date:

ACKNOWLEDGEMENT

The needs and deeds of a particular person are only satisfied with the support and endurance of many. I take this opportunity to thank almighty first for giving me Health and Patience in completing my Project.

I would like to extend my heartfelt thanks to Chairman Sri C. Gangi Reddy for his constant encouragement and support during the Project period.

I would like to extend my heartfelt thanks to Principal Dr. C. Nadhamuni Reddy for his constant encouragement and support during the Project period.

I would like to express my heartfelt thanks to Ms. B. Rupa Devi, M.Tech(Ph.D), Associate Professor and HOD, Department of AI&DS during the progress of project for her timely suggestions and help in spite of her busy schedule.

I would like to extend my heartfelt thanks to our department Project Coordinator Mr. K. Jayachandra, M. Tech, Assistant Professor, Department of AI&DS for providing consistent support for us to complete the course modules in order to complete my project.

My heartfelt thanks to Project guide Ms. A. Sathvika, M.Tech, Assistant Professor, Department of AI&DS for her valuable guidance and suggestions in analysis and testing throughout the period, till the end of project work completion.

I am thankful to the Project Review Committee who have invested their valuable time to conduct our monthly presentations and provided their feedback with a lot of useful suggestions.

I also thank all the Faculty Members, of the Department of Annamacharya Institute of Technology and Sciences.

T DILEEP	22AK1A3022
D LAKSHITHA	22AK1A3042
B AKILA	22AK1A3005
L GAYATHRI	22AK1A3027

ABSTRACT

QuizGenix is an intelligent web-based system designed to automate quiz generation and evaluation using advanced Generative Artificial Intelligence. In traditional educational environments, creating quizzes and evaluating student performance requires significant manual effort and time. To address this challenge, the proposed system utilizes the Gemini Large Language Model (LLM) to generate high-quality multiple-choice questions directly from educational materials such as PDF documents.

The system extracts textual content, processes it using feature engineering techniques such as chunking and semantic embeddings, and generates context-aware questions through prompt-based interactions with the LLM. To ensure diversity and quality, duplicate questions are filtered using similarity analysis based on cosine similarity. The platform also provides automated evaluation of student responses and maintains performance records for further analysis.

Additionally, QuizGenix supports adaptive learning by generating personalized quizzes based on student performance, helping learners focus on weak areas. The system is implemented using Python and Django, with integration of LangChain and Gemini API for AI functionality.

Overall, QuizGenix reduces instructor workload, improves assessment efficiency, and enhances the learning experience by providing intelligent, scalable, and automated quiz generation and evaluation.

Keywords - Quiz Generation, Generative AI, Large Language Models (LLMs), Gemini, Natural Language Processing (NLP), Semantic Embeddings, Feature Engineering, Automated Evaluation, Adaptive Learning, Educational Technology, Django, LangChain

TABLE CONTENTS

CONTENTS	PAGE NO
CHAPTER – 1 : INTRODUCTION	
1.1 Objective	2
1.2 Plan of Action	2
1.3 Literature Survey	4
1.3.1 Comparison Table	7
1.4 Business Context	7
1.4.1 Existing System Study	8
1.5 Proposed System	9
1.6 System Requirements	11
1.7 Functional and Non – Functional Requirements	12
1.8 Feasibility Study	13
CHAPTER – 2 : DATASET DESCRIPTION	
2.1 Data Sources	15
2.2 Data Description	16
2.3 Data Cleaning/Preprocessing	17
2.3.1 Handling missing values	18
2.3.2 Data Filtering And Quality Control	18
2.3.3 Data transformations	19
CHAPTER – 3 : EXPLORATORY DATA ANALYSIS (EDA)	
3.1 Key Insights	21
3.2 Visualizations	22
3.2.1 Correlations	22
3.2.2 Distribution Plots	23
3.2.3 Anomaly Detection	24
CHAPTER 4 - FEATURE ENGINEERING	
4.1 Feature Selection	27
4.2 New Feature Selection Creation	29
4.3 Feature Scaling	31

CHAPTER – 5 : MODEL DEVELOPMENT	
5.1 Algorithms Considered	35
5.2 Model Selection	36
5.3 Hyperparameter Tuning	37
5.4 Training Process	38
5.4.1 Train/test split	39
5.4.2 Cross Validation	39
CHAPTER – 6 MODEL EVALUATION AND TESTING	
6.1 Evaluation Metrics	42
6.2 Performance Results	44
6.3 Model Comparison	45
6.4 Error Analysis	46
CHAPTER – 7 : DEPLOYMENT STRATEGY	
7.1 Model Serialization	47
7.2 Deployment Tools /Frameworks	48
7.3 Integration Processes	49
7.4 Output Screenshots	51
CHAPTER – 8 : MONITORING AND MAINTENANCE	
8.1 Performance Tracking	59
8.2 Retraining Strategy	60
8.3 Monitoring Tools	62
CHAPTER – 9 : CONCLUSION	63
CHAPTER – 10 : FUTURE SCOPE OF THE PROJECT	64
CHAPTER – 11 : APPENDIX	
11.1 Code Repository	65
11.1.1 Github links	65

11.1.2 Complete Project Code	65
11.2 List of Abbreviations and Symbols Used	82
11.3 Publications Certificates/Awards	83
11.4 Published Paper	93
11.5 References/Bibliography	94

LIST OF FIGURES

Figure No	Name	Page No
7.4	Output Screenshots	51
7.4.1	Home Page	51
7.4.2	Registration for General user	52
7.4.3	Login Page	52
7.4.4	Teacher Registration Page	53
7.4.5	Teacher Login	53
7.4.6	OTP Verification	54
7.4.7	General User Dashboard	54
7.4.8	Teacher dashboard	55
7.4.9	Student upload page	55
7.4.10	Generated student credentials	56
7.4.11	AI quiz creation page	56
7.4.12	Quiz creation page	57
7.4.13	General user dashboard options	57
7.4.14	Quiz attempt interface	58
7.4.15	Quiz Result	58

CHAPTER-1

INTRODUCTION

The rapid advancement of Artificial Intelligence (AI) and Natural Language Processing (NLP) has significantly transformed many fields, including education. Modern educational environments increasingly rely on digital platforms to manage learning materials, assessments, and student performance. Among these components, quizzes and assessments play a crucial role in evaluating student understanding and reinforcing learning outcomes. However, creating high-quality quizzes manually is a time-consuming process for educators, especially when dealing with large classes or frequently updated course materials. Traditionally, instructors must carefully read educational content, identify key concepts, formulate multiple-choice questions (MCQs), create plausible distractors, and evaluate student responses. This process demands significant time and effort. In large classrooms or online learning environments, the manual generation and evaluation of quizzes becomes inefficient and difficult to scale. Recent advancements in Large Language Models (LLMs) have introduced new opportunities for automating educational content generation. LLMs such as Gemini possess strong capabilities in understanding natural language, summarizing text, and generating structured content such as questions, explanations, and summaries. By leveraging these capabilities, it is possible to automatically generate assessment questions directly from educational materials such as textbooks, lecture notes, or PDF documents. The project QuizGenix – Automated Quiz Generation System using Gemini and LangChain is designed to address these challenges by providing an AI-powered platform for generating and evaluating multiple-choice quizzes from educational materials. The system allows instructors or users to upload educational PDF documents or provide textual content. The system extracts text from the uploaded documents and processes it using a pretrained Gemini large language model through the Gemini API integrated with LangChain. Once the content is processed, the system automatically generates multiple-choice questions with four options and identifies the correct answer along with a short explanation. To ensure that the generated questions are diverse and meaningful, the system incorporates semantic duplicate filtering using sentence transformer embeddings. This helps eliminate repetitive or highly similar questions that may arise from similar text segments.

In addition to automated quiz generation, QuizGenix includes several features designed to support educational workflows. Teachers can upload Excel files containing student information

to automatically create student accounts. Students can attempt quizzes through a web interface built with Django, and the system evaluates responses automatically. Each quiz can optionally include a countdown timer to support time-bound assessments. The system also tracks student performance at the question level. Using historical performance records, QuizGenix can generate personalized quizzes that focus on areas where a student previously answered incorrectly. This enables targeted practice and supports adaptive learning approaches. The architecture of the system integrates multiple technologies including Python, Django, LangChain, Gemini API, and sentence-transformer embeddings. Django provides the web application framework, while LangChain facilitates communication with the Gemini model. The Gemini LLM is responsible for generating the MCQs based on extracted educational content. By combining modern AI capabilities with a structured web application framework, QuizGenix aims to streamline quiz creation, reduce administrative workload for instructors, and improve the learning experience for students.

1.1 Objective

The primary objective of the QuizGenix system is to develop an AI-powered platform capable of automatically generating and evaluating multiple-choice quizzes from educational materials. The system aims to simplify the quiz creation process for instructors while providing students with a convenient platform for practicing and assessing their knowledge.

The specific objectives of the project include:

- i. To develop a web-based system that allows users to upload educational PDF documents or input text.
- ii. To extract textual content from uploaded documents using PDF text extraction libraries.
- iii. To integrate the Gemini large language model through the Gemini API and LangChain for generating multiple-choice questions.
- iv. To automatically generate MCQs with four answer options and identify the correct answer along with explanations.
- v. To implement semantic duplicate filtering using sentence transformer embeddings to eliminate repetitive questions.

- vi. To enable students to attempt quizzes through an interactive web interface built using Django.
- vii. To implement timer-based quiz functionality for time-controlled assessments.
- viii. To track student performance and maintain records of quiz attempts.
- ix. To generate personalized quizzes based on students' previous incorrect answers.
- x. To allow teachers to upload Excel files to automatically create multiple student accounts.

1.2 Plan Of Action

The development of the QuizGenix system follows a structured approach consisting of several stages that transform educational content into automatically generated quizzes. The workflow begins with user interaction through the web interface and continues through multiple processing steps until quiz evaluation and performance tracking are completed.

The plan of action for the system can be summarized as follows:

Step 1: UserInteraction and Document Upload

Teachers or users access the system through the Django web application and upload educational PDF documents or provide textual content.

Step 2: Text Extraction from Documents

The uploaded PDF documents are processed using Python libraries such as pdfplumber or pypdf. These libraries extract textual content from the document pages.

Step 3: Text Cleaning and Processing

The extracted text is cleaned to remove unwanted characters and formatting issues. The text is then divided into manageable chunks to ensure efficient processing by the language model.

Step 4: AI-Based Question Generation

Each text chunk is sent to the Gemini large language model through the Gemini API using LangChain. The model processes the content and generates multiple-choice questions based on the input text.

Step 5: Duplicate and Similarity

Filtering Generated questions are analyzed using sentence-transformer embeddings. Semantic similarity calculations are performed to detect duplicate or highly similar questions. Redundant questions are removed.

Step 6: Quiz Creation

The filtered questions are stored in the database and associated with a specific quiz created by the instructor.

Step 7: Student Quiz Attempt

Students log in to the system and attempt quizzes through the web interface. Timer settings may restrict the duration of the quiz.

Step 8: Automatic Evaluation

Student responses are automatically compared with stored correct answers, and scores are calculated instantly.

Step 9: Performance Tracking

The system stores detailed performance records for each question attempt.

Step 10: Personalized Quiz Generation

Based on incorrect responses recorded in previous quizzes, the system generates personalized quizzes to help students focus on weak areas.

1.3 Literature Survey

i. Automatic Question Generation from Text Using NLP

Authors: Michael Heilman, Noah A. Smith **Published:**

2010, Carnegie Mellon University **Abstract:**

This work presents an approach for automatically generating questions from textual data using Natural Language Processing techniques. The system focuses on transforming declarative sentences into interrogative forms by applying syntactic and semantic analysis. It is capable of generating factual and comprehension-based questions from educational content. However, the approach relies heavily on rule-based methods and lacks contextual understanding, limiting the

quality and diversity of generated questions. The proposed project builds upon this limitation by incorporating Generative AI models to produce more intelligent, context-aware, and diverse questions.

The approach successfully converts declarative sentences into questions using rule-based NLP techniques. However, it lacks deep contextual understanding and produces limited variation in questions. These limitations motivate the use of Generative AI to create more diverse, context-aware, and meaningful questions.

ii. Language Models are Few-Shot Learners

Authors: Tom B. Brown et al.

Published: 2020, NeurIPS / arXiv

Abstract:

This paper introduces large-scale language models capable of performing multiple natural language processing tasks using few-shot learning. The authors demonstrate that models like GPT can generate high-quality text, answer questions, and perform reasoning tasks without task-specific training. This research highlights the potential of Generative AI in automating complex tasks such as question generation and evaluation. However, the training of such models requires extensive computational resources. The proposed project leverages pre-trained models through APIs to enable efficient quiz generation and automated evaluation without requiring heavy infrastructure.

iii. Neural Question Generation Using Sequence-to-Sequence Models

Authors: Xinya Du, Junru Shao, Claire Cardie

Published: 2017, ACL Conference **Abstract:**

This study proposes a neural network-based approach for question generation using sequence-to-sequence learning with attention mechanisms. The model generates questions from input passages by learning contextual relationships between words and sentences. It significantly improves over rule-based systems by producing more natural and relevant questions. However, it still requires large labeled datasets and struggles with generating complex or analytical questions. The proposed project enhances this approach by using advanced Large Language Models that can

generate higher-quality, context-aware, and diverse questions without requiring extensive labeled data.

iv. Automatic Short Answer Grading Using Machine Learning

Authors: Mohler, Bunescu, Mihalcea

Published: 2011, ACL **Abstract:**

This research focuses on automated evaluation of short answers using machine learning techniques. The system compares student responses with reference answers using semantic similarity measures to assign scores. It reduces manual grading effort and improves evaluation consistency. However, the approach is limited in handling subjective answers and lacks deep contextual understanding. The proposed project improves upon this by using Generative AI models capable of understanding context, intent, and semantic meaning, enabling more accurate and flexible evaluation of student responses.

v. BERT: Pre-training of Deep Bidirectional Transformers for Language

Understanding

Authors: Jacob Devlin et al. **Published:**

2018, Google AI / NAACL **Abstract:**

This paper introduces BERT, a transformer-based model designed for deep bidirectional language understanding. BERT achieves state-of-the-art performance in various NLP tasks, including question answering and text classification. It demonstrates strong contextual understanding by analyzing text from both directions. However, BERT is primarily designed for understanding rather than generation tasks. The proposed project extends beyond this limitation by utilizing generative models capable of both understanding and generating questions, as well as evaluating answers intelligently.

Furthermore, generative models enable dynamic content creation, allowing the system to produce diverse and context-aware questions from the same input. This enhances adaptability and supports more interactive and personalized learning experiences compared to traditional static NLP models.

1.3.1 Comparison Table

S.No.	System	TechnologyUsed	Method	Limitations
1.	Templatebased AQG Systems	Rule-based NLP	Predefined templates generate questions	Limited flexibility
2.	Neural Question Generation	Deep Learning Models	Sequencetosequence generation	Requires training datasets
3.	Transformerbased AQG	BERT, T5, GPT	Contextaware question generation	Requires training datasets
4.	LLM-based Systems	ChatGPT/ Gemini	Promptbased question generation	Requires prompt engineering
5.	QuizGenix (Proposed System)	Gemini LLM + LangChain	AIgenerated MCQs from educational PDFs	Depends on API access

1.4 Business Context / Impact

The education sector is undergoing a rapid digital transformation, driven by the increasing need to manage large student populations, diverse learning resources, and scalable assessment systems. Institutions today require intelligent solutions that not only streamline academic processes but also enhance the overall learning experience. One of the critical areas in this transformation is **assessment management**, particularly quiz and test generation.

Traditional methods of assessment creation are labor-intensive and time-consuming. Educators must carefully analyze course materials, identify key learning outcomes, and design questions that

effectively evaluate student understanding. This process becomes significantly more challenging when dealing with multiple courses, large class sizes, or frequently updated syllabi. As a result, instructors often face workload pressure, which can impact the quality and frequency of assessments.

Automating quiz generation presents a practical solution to these challenges. By leveraging artificial intelligence, particularly Large Language Models (LLMs), it becomes possible to generate high-quality questions directly from educational content. This reduces manual effort while maintaining consistency, relevance, and coverage of important topics.

QuizGenix is designed to address these challenges by integrating AI-driven quiz generation with a web-based learning management system. The platform enables instructors to upload study materials, from which quizzes are automatically generated. This significantly reduces preparation time and allows educators to focus more on teaching and student engagement rather than administrative tasks.

Furthermore, QuizGenix enhances learning outcomes through **adaptive learning capabilities**. By analyzing student performance data, the system generates personalized quizzes tailored to individual strengths and weaknesses. This targeted approach helps students focus on areas that require improvement, thereby increasing learning efficiency and retention.

From an institutional perspective, the system improves scalability, consistency in assessments, and data-driven decision-making. It also supports modern educational practices such as continuous assessment, self-paced learning, and personalized education, making it highly relevant in today's digital learning environment.

1.4.1 Existing System Study

In traditional educational environments, quiz creation is primarily a manual process. Instructors review course materials, identify important concepts, and design questions along with appropriate answer choices. This process requires considerable time and effort, especially when assessments need to be updated frequently or customized for different student groups.

Even with the adoption of digital learning management systems (LMS), the process remains largely manual. Most existing systems provide interfaces for creating and managing quizzes, but they do not support automatic question generation. Educators must still input each question, define

answer options, and configure evaluation criteria. This limits efficiency and does not significantly reduce instructor workload.

Another limitation of existing systems is the lack of **personalization**. Students typically receive the same set of questions regardless of their individual performance levels or learning needs. This one-size-fits-all approach reduces the effectiveness of assessments, as it does not adapt to varying levels of understanding among students.

Additionally, manual systems often lack advanced analytics and intelligent feedback mechanisms. While basic scoring is available, deeper insights into student performance—such as identifying weak areas or recommending targeted practice—are generally absent. This limits the ability of educators to provide personalized guidance and support.

Overall, the existing systems are:

- Time-consuming and labor-intensive
- Limited in scalability
- Lacking automation and intelligence
- Ineffective in providing personalized learning experiences

These limitations highlight the need for an advanced, AI-driven solution that can automate quiz generation and enhance assessment quality.

Moreover, manual quiz design often leads to repetition and limited question diversity across assessments. It also increases the chances of human error in framing questions and setting evaluation criteria. An AI-driven approach can streamline this process while ensuring consistency, variety, and improved assessment effectiveness.

1.5 Proposed System

The proposed system, QuizGenix, introduces an intelligent and automated approach to quiz generation and evaluation using advanced Large Language Models, specifically the **Gemini LLM**. The system is designed to minimize manual effort while maximizing efficiency, accuracy, scalability, and adaptability in the assessment process, making it suitable for modern digital learning environments.

In this system, instructors can upload educational materials such as PDFs containing lecture notes, textbooks, or study guides, or directly input textual content. The system processes these documents by extracting textual data using robust parsing techniques and applying natural language understanding methods to interpret the content. With the help of **LangChain**, structured prompts are generated and sent to the Gemini model, which produces high-quality, context-aware multiple-choice questions aligned with the learning material.

To ensure the quality, relevance, and diversity of generated questions, the system incorporates a semantic duplicate filtering mechanism using sentence-transformer embeddings. This technique compares the semantic similarity between questions and removes redundant or highly similar ones, thereby maintaining uniqueness and improving the overall effectiveness of assessments. Additionally, difficulty levels can be varied to create a balanced set of questions suitable for different learning stages.

Once the quizzes are generated, they can be assigned to students through an intuitive and responsive web interface developed using **Django**. The platform supports seamless quiz management features such as scheduling, timed assessments, attempt restrictions, and real-time submission tracking. Student responses are evaluated automatically, and instant feedback is provided, including correct answers and explanations, which helps reinforce learning.

A key feature of QuizGenix is its adaptive learning capability. The system continuously monitors and records student performance at a granular level, identifying patterns in strengths and weaknesses across topics and question types. Based on this analysis, it dynamically generates personalized quizzes tailored to individual learning needs, ensuring targeted practice and progressive improvement. Over time, this adaptive mechanism enhances concept retention, boosts confidence, and supports self-paced learning.

Furthermore, the system provides detailed performance analytics through dashboards that present insights such as accuracy trends, topic-wise performance, and time taken per question. These insights enable educators to make data-driven decisions, identify struggling students, and adjust teaching strategies accordingly. The integration of AI-driven automation with a structured web framework not only reduces administrative workload but also creates a more engaging, efficient, and personalized learning experience for students.

1.6 System Requirements

Hardware Requirements

Requirement	Description
Processor	Intel i5 or equivalent processor for efficient system performance
RAM	Minimum 8 GB RAM to support smooth multitasking and AI processing
Storage	At least 256 GB storage for application files, datasets, and user data

Software Requirements

Requirement	Description
Operating System	Windows / Linux / macOS for cross-platform compatibility
Programming Language	Python for backend development and AI integration
Web Framework	Django for building a scalable and secure web application
AI Integration	LangChain for managing LLM workflows and prompt handling
LLM Model	Gemini 1.5 Flash for fast and efficient quiz generation
Database	SQLite for lightweight data storage and management
Libraries	pdfplumber, pypdf for text extraction; sentence-transformers for semantic processing; openpyxl for Excel handling

1.7 Functional and Non-functional Requirements

Functional Requirements

Requirement	Description
User Registration and Login	The system should allow users to create accounts and securely log in to access the platform
PDF Upload	The system should allow instructors to upload educational PDF documents
Text Extraction	The system should extract textual content from uploaded PDF files for processing
MCQ Generation	The system should generate multiple-choice questions using the Gemini LLM
Duplicate Filtering	The system should remove duplicate or semantically similar questions to ensure diversity
Quiz Creation	The system should allow creation of quizzes from generated questions
Quiz Attempt	The system should allow students to attempt quizzes through the web interface
Timer-Based Quiz	The system should provide time-bound quiz functionality
Automatic Evaluation	The system should automatically evaluate student responses and calculate scores
Performance Tracking	The system should track and store student performance data

Non-Functional Requirements

Requirement	Description
Performance	The system should generate quizzes and responses within a reasonable time for smooth user experience
Security	The system should ensure secure user authentication and protect sensitive data
Usability	The system interface should be user-friendly and easy to navigate
Scalability	The system should handle multiple users simultaneously without performance issues
Reliability	The system should store and manage student performance data accurately and consistently
Maintainability	The system should be developed with modular and readable code for easy updates and maintenance

1.8 Feasibility Study

Before implementing the system, it is important to evaluate its feasibility from different perspectives

i **Technical Feasibility**

The proposed system, QuizGenix, is technically feasible as it is built using well-established and widely supported technologies such as Python, Django, and the Gemini API. Python provides a robust ecosystem for implementing Natural Language Processing (NLP) and integrating Large Language Models (LLMs), while Django offers a secure and scalable framework for developing web-based applications.

The integration of the Gemini 1.5 Flash model enables efficient and high-quality generation of multiple-choice questions from textual content. Additionally, tools like LangChain simplify

interaction with LLMs, making it easier to design prompt workflows and manage responses. Libraries such as pdfplumber and pypdf support accurate text extraction from uploaded documents, while sentence-transformers enable semantic similarity checks for duplicate question filtering.

ii **Economic Feasibility**

The proposed system is economically feasible as it relies primarily on open-source technologies and frameworks, significantly reducing development costs. Tools such as Python, Django, and supporting libraries are freely available, eliminating licensing expenses.

The primary cost component of the system is the usage of the Gemini API, which operates on a usage-based pricing model. However, since the system is intended for educational purposes and moderate usage, the overall operational cost remains relatively low and manageable. Additionally, the system reduces indirect costs by:

Minimizing instructor workload in quiz creation

Reducing time spent on manual evaluation

Improving efficiency in managing assessments

The scalability of the system also ensures that it can be expanded gradually without requiring significant upfront investment. Institutions can start with minimal infrastructure and scale based on demand.

iii **Operational Feasibility**

The system is operationally feasible as it is designed with a user-friendly and intuitive web interface, ensuring ease of use for both instructors and students. The platform simplifies complex processes such as quiz generation, assignment distribution, and performance evaluation into a few straightforward steps. Key features that enhance operational efficiency include:

Automated quiz generation from uploaded PDFs

Bulk student upload using Excel files

Instant evaluation and score generation

Performance tracking and analytics

Chapter 2

Dataset Description

The performance and effectiveness of any Artificial Intelligence system largely depend on the quality and structure of the data being processed. In the case of the QuizGenix system, the concept of a traditional static dataset does not apply because the system does not rely on a fixed dataset collected during development. Instead, the system dynamically processes educational content uploaded by users or instructors.

QuizGenix operates on educational documents provided by users in PDF format or plain text input. These documents typically contain lecture notes, study materials, textbooks, or instructional content from which the system automatically generates multiple-choice questions.

The uploaded documents serve as the primary input data source for the system. The content within these documents is extracted, processed, and analyzed by the Gemini large language model to generate relevant quiz questions.

This dynamic data processing approach allows the system to adapt to different subjects, educational domains, and learning materials without requiring retraining or a predefined dataset.

2.1 Data Sources

The QuizGenix system obtains its data from user-provided educational materials. These materials are uploaded directly through the web application interface by instructors or general users.

The main sources of data include:

1. Educational PDF Documents

The primary data source for the system consists of PDF documents uploaded by users. These documents may include: i Lecture notes ii Textbook chapters iii Study guides iv Educational articles

These documents contain structured educational content that the system processes to generate multiple-choice questions.

2. Direct Text Input

In addition to uploading PDF files, the system also allows users to paste educational text directly into the quiz generation interface. This enables users to generate quizzes from smaller sections of study material without uploading an entire document.

3. Personalized Quiz Context

The system also uses internal data generated from student performance records. When students attempt quizzes, their incorrect responses are recorded. These records are later used to generate personalized quizzes that focus on weak areas.

Therefore, the QuizGenix system processes two main types of data:

- Educational content data (from PDFs or text input)
- Student performance data (from quiz attempts)

2.2 Data Description

The data processed by QuizGenix consists of natural language educational text extracted from uploaded documents.

This text typically contains:

Definitions

Explanations

Concept descriptions

Technical terminology

Examples and instructional content

The structure of the extracted text depends on the formatting of the original PDF document. Some documents contain well-organized paragraphs, while others may include bullet points, headings, tables, or diagrams.

Once extracted, the textual content is treated as the context input for the Gemini LLM. The language model analyzes the content and generates multiple-choice questions that reflect the key concepts described in the text.

Each generated MCQ typically includes the following components:

Component	Description
Question	The main question derived from the educational text
Option A	First possible answer
Option B	Second possible answer
Option C	Third possible answer
Option D	Fourth possible answer

Correct Answer The correct option identified by the system Explanation

A short explanation of the correct answer

These structured question objects are stored in the system database and associated with specific quizzes created by instructors.

2.3 Data Cleaning / Preprocessing

Before the educational text is sent to the Gemini LLM for question generation, it undergoes several preprocessing steps. These steps help ensure that the language model receives clean and wellstructured input text.

The preprocessing pipeline includes:

Text Extraction

Text Cleaning

Text Chunking

Formatting and Normalization

These preprocessing steps improve the quality of generated questions by reducing noise in the input data.

Before the educational text is sent to the **Gemini** LLM for question generation, it undergoes several preprocessing steps to ensure clean and well-structured input.

2.3.1 Handling Missing Values

Unlike structured datasets, the educational text processed by QuizGenix does not contain traditional missing values such as empty numeric fields or null attributes. However, missing information can still occur in the form of: Empty pages in PDFs

Pages containing only images

Documents with very little textual content

To address these situations, the system performs validation checks during the text extraction stage. If a page contains no extractable text, the system simply ignores that page and continues processing the remaining content.

Additionally, the system ensures that only meaningful text segments are passed to the language model.

Empty or extremely short text blocks are filtered out before question generation.

This process ensures that the Gemini LLM receives sufficient contextual information to generate meaningful questions.

2.3.2 Outlier Detection and Treatment

In the context of QuizGenix, outliers refer to irrelevant or noisy text segments that do not contribute to meaningful question generation.

Examples of such outliers include:

Page numbers

Headers and footers

Repeated titles

Formatting artifacts

Special characters

These elements may appear during PDF text extraction and can negatively affect the quality of generated questions.

To reduce the impact of such noise, the system performs basic text cleaning operations such as:

Removing excessive whitespace

Eliminating unnecessary line breaks

Filtering special characters where necessary

Normalizing text formatting

By removing these textual artifacts, the system ensures that the input provided to the Gemini LLM is clear and meaningful.

2.3.3 Data Transformations

After cleaning the extracted text, the system performs a critical transformation step known as text chunking.

Large language models have input size limitations, meaning they cannot process extremely large text inputs in a single request. Therefore, long documents must be divided into smaller segments before being processed.

The QuizGenix system divides the extracted text into overlapping text chunks.

The chunking process works as follows:

The cleaned text is divided into segments of a fixed size.

Each segment contains a portion of the document's content.

Adjacent segments overlap slightly to preserve contextual continuity.

For example:

Chunk Content

Chunk 1 Paragraphs 1–3

Chunk 2 Paragraphs 3–5

Chunk 3 Paragraphs 5–7

This overlapping approach ensures that important contextual information is not lost when dividing the document.

Each chunk is passed to the **Gemini** LLM via **LangChain**.

Chapter 3

Exploratory Data Analysis (EDA)

Exploratory Data Analysis (EDA) is an essential step in understanding the characteristics and structure of data before applying computational models or processing pipelines. In the QuizGenix system, EDA is performed on the **educational text extracted from uploaded PDF documents and the generated multiple-choice questions (MCQs)**.

Unlike traditional machine learning systems that analyze structured datasets containing numeric attributes, QuizGenix processes **unstructured textual data** derived from educational materials. Therefore, the goal of exploratory analysis in this project is to understand the characteristics of the extracted text, evaluate the distribution of generated questions, and identify patterns or anomalies that may influence quiz generation quality.

EDA in QuizGenix focuses on several aspects of the system's data pipeline, including:

- Analysis of extracted textual content from uploaded PDFs
- Distribution of educational concepts across text segments
- Examination of generated MCQs
- Detection of duplicate or highly similar questions
- Identification of irregular text segments or anomalies

By performing exploratory analysis on the extracted text and generated questions, the system ensures that the AI-generated quizzes remain meaningful, diverse, and aligned with the educational content.

EDA also evaluates sentence length, keyword frequency, and readability levels of the extracted content to ensure suitability for question generation. It helps in identifying noisy text such as headers, footers, or formatting artifacts that may affect processing quality. The analysis further checks the balance of topics to avoid overrepresentation of certain concepts in the quiz.

Visualization of question patterns aids in understanding the diversity and coverage of generated MCQs. These insights guide improvements in preprocessing and prompt design for more accurate and relevant quiz creation.

3.1 Key Insights

During the development and evaluation of QuizGenix, several key observations were made regarding the nature of the extracted educational content and the generated quiz questions. One important observation is that educational documents typically contain structured explanatory text. These documents often include definitions, conceptual explanations, examples, and procedural descriptions. Such content is highly suitable for generating multiple-choice questions because it contains clear conceptual statements that can be transformed into question-answer pairs. Another key insight is that large documents often contain repeated concepts or similar explanations. For example, different sections of a document may describe related ideas using slightly different wording. When the language model processes such text segments independently, it may generate questions that are semantically similar.

To address this issue, the system incorporates semantic similarity filtering using `sentence-transformer` embeddings. This filtering mechanism ensures that questions with similar meanings are detected and removed, improving quiz diversity.

Additionally, the analysis of generated MCQs revealed that most questions correspond to fundamental concepts within the educational text. This indicates that the Gemini large language model is capable of identifying important informational elements within the source material.

Another important observation is that long educational documents contain varying levels of detail. Some sections may contain dense conceptual information, while others include examples or extended explanations. Dividing the document into smaller chunks allows the system to process each section independently while preserving contextual relevance.

These insights guided the design of the text processing pipeline and contributed to the development of effective MCQ generation workflows.

Further observation showed that preprocessing steps such as text cleaning and normalization significantly improve the quality of generated questions. It was also noted that chunk size plays a crucial role in maintaining context without overwhelming the language model. Monitoring the proportion of questions per chunk helps ensure balanced coverage of the entire document. Evaluation of distractor options revealed the need for clarity and plausibility to maintain assessment quality. These findings informed iterative refinements to the generation, filtering, and validation stages of the system.

3.2 Visualizations

Visualizations are commonly used in exploratory data analysis to represent patterns, relationships, and distributions within the data. In the context of QuizGenix, visualizations help illustrate the structure of the educational text and the characteristics of generated quiz questions.

Although the system primarily processes textual data, conceptual visualizations can still be used to represent different aspects of the quiz generation pipeline.

Key areas for visualization include:

- Text chunk distribution
- Question generation pipeline
- Similarity filtering process
- Question diversity analysis

These visualizations help illustrate how the system processes educational content and ensures question quality.

3.2.1 Correlations

In traditional datasets, correlation analysis measures relationships between numeric variables. However, in QuizGenix the concept of correlation applies to **semantic relationships between generated questions**. Questions generated from the same educational document may exhibit correlations if they reference similar concepts or topics. For example, two different text segments describing the same concept may produce questions with similar meanings.

To measure these semantic correlations, QuizGenix uses **sentence-transformer embeddings**. Each generated question is converted into a vector representation that captures its semantic meaning. These vectors allow the system to measure similarity between questions using cosine similarity.

The cosine similarity score between two question embeddings indicates how closely related the questions are in terms of semantic meaning.

The similarity score ranges from:

Similarity Score Interpretation

0.0 - 0.4	Low similarity
0.4 - 0.7	Moderate similarity
0.7 - 1.0	High similarity

If the similarity score between two questions exceeds a predefined threshold (for example 0.85), the system considers them semantically redundant and removes one of them from the quiz. This process helps ensure that generated quizzes contain diverse questions covering different aspects of the educational material.

3.2.2 Distribution Plots

Distribution analysis helps understand how educational content and generated questions are distributed across different sections of the input documents.

In QuizGenix, distribution analysis focuses on the following elements:

Text Chunk Distribution

When a PDF document is uploaded, the extracted text is divided into multiple chunks. Each chunk represents a segment of the document that is processed independently by the Gemini LLM.

For example:

Chunk Number Text Size

Chunk 1	4000 characters
Chunk 2	4000 characters
Chunk 3	3800 characters
Chunk 4	4000 characters

This distribution ensures that each text segment remains within the token limits supported by the language model.

Question Distribution

Each chunk may generate several MCQs depending on the configuration specified in the prompt template.

For example:

Chunk Questions Generated

Chunk 1 5 questions

Chunk 2 5 questions

Chunk 3 4 questions

Chunk 4 5 questions

The total number of questions collected from all chunks forms the candidate question set before duplicate filtering.

This distribution helps ensure that the quiz covers different sections of the document rather than focusing on only a small portion of the content.

3.2.3 Anomaly Detection

Anomaly detection is used to identify unusual or problematic data patterns that may negatively impact the quality and reliability of generated quizzes. In QuizGenix, this process act as a validation layer within the preprocessing and generation pipeline, ensuring that only meaningful and well-structured data is passed forward for question creation.

In QuizGenix, anomalies may occur in several forms, such as incomplete or low-content text segments, repetitive or duplicate information, formatting inconsistencies introduced during PDF extraction, and overly complex or ambiguous sentences. By detecting and handling these irregularities early, the system prevents the generation of unclear, redundant, or low-quality

questions. This ultimately enhances the accuracy, diversity, and effectiveness of the quizzes, leading to a better learning and assessment experience for students.

Empty or Very Short Text Segments

If a PDF page contains very little extractable text, it may not provide enough contextual information for generating meaningful and accurate questions. Such low-content segments can lead to vague, incomplete, or irrelevant MCQs. To prevent this, the system filters out these segments during the chunking stage and processes only those portions that contain sufficient informational depth. This ensures that the generated questions are contextually rich, clearly framed, and aligned with the source material, ultimately improving the overall quality of the quiz.

If a PDF page contains very little extractable text, it may not provide enough contextual information for generating meaningful and accurate questions. Such low-content segments can lead to vague, incomplete, or irrelevant MCQs. To prevent this, the system filters out these segments during the chunking stage and processes only those portions that contain sufficient informational depth. This ensures that the generated questions are contextually rich, clearly framed, and aligned with the source material, ultimately improving the overall quality of the quiz..

Formatting Artifacts

PDF extraction can introduce artifacts such as broken sentences, irregular spacing, or isolated symbols. These issues are handled through text cleaning and normalization processes that reconstruct sentences and remove unnecessary characters, improving readability and model interpretation.

Extremely Complex Sentences: Technical or academic documents often contain long and complex sentences that are difficult to convert into clear and concise questions. The chunking process breaks such content into smaller, manageable segments, enabling better comprehension by the model and more accurate question generation. Additionally, the system incorporates validation checks to ensure that the processed text meets minimum quality standards before being sent to the **Gemini LLM**

Chapter 4

Feature Engineering

Feature engineering is an essential stage in Artificial Intelligence systems, as it transforms raw input data into a structured form that can be effectively understood and processed by computational models. In traditional machine learning, this process typically involves selecting important features, creating new derived attributes, and normalizing data to improve model performance.

In the **QuizGenix system**, feature engineering is applied to **textual data** extracted from educational documents such as PDFs. Unlike numerical datasets, textual data is unstructured and requires careful preprocessing before it can be used by AI models. Therefore, feature engineering in this system focuses on converting raw text into meaningful and structured representations that can be efficiently processed by the **Gemini Large Language Model (LLM)**.

The primary objective of feature engineering in QuizGenix is to ensure that the input text is well-organized, contextually rich, and suitable for generating high-quality quiz questions. To achieve this, several important transformations are applied:

- **Text Cleaning and Preprocessing:**

The extracted text is cleaned by removing unnecessary symbols, extra spaces, and irrelevant content. This improves the clarity and quality of the input.

- **Chunk Creation:**

Large documents are divided into smaller, manageable segments or chunks. This helps the model focus on specific portions of content and improves the relevance of generated questions.

- **Semantic Embedding Generation:**

Each text chunk is converted into vector representations using embedding techniques. These embeddings capture the semantic meaning of the content, allowing the system to understand relationships between different parts of the text.

- **Duplicate Detection:**

By comparing embeddings, the system identifies and removes semantically similar or duplicate questions. This ensures diversity and avoids repetition in the generated quizzes.

- **Structured Question Formatting:**

The processed text is transformed into a structured format suitable for quiz generation, such as multiple-choice questions with options and correct answers.

These feature engineering steps enable the system to move beyond simple text processing and achieve deeper contextual understanding. As a result, QuizGenix can generate **accurate, diverse, and meaningful questions** that cover multiple topics within the educational material.

4.1 Feature Selection / Extraction

Feature selection refers to the process of identifying the most relevant information from the raw data that will be used for further processing.

In the QuizGenix system, the input data consists of educational text extracted from uploaded PDF documents. The system identifies key informational segments within the text that can be used as context for generating questions.

The primary features extracted from the raw text include:

Educational Text Content

The most important feature used by the system is the educational content extracted from the uploaded PDF document. This content contains the concepts, explanations, and definitions that form the basis for question generation.

The text extraction module reads each page of the document and collects the textual content into a continuous text sequence. This text is then cleaned and prepared for further processing. Text Segments (Chunks)

Large language models have limitations on the maximum size of input text they can process in a single request. Therefore, long documents must be divided into smaller segments.

The system extracts **text chunks** from the full document. Each chunk represents a section of the educational material that will be processed independently by the Gemini LLM. These chunks serve as contextual features that guide the question generation process.

The most important feature used by the system is the educational content extracted from the uploaded PDF document. This content contains the concepts, explanations, and definitions that form the basis for question generation.

Question Structure Components

When the Gemini LLM generates questions, the output is structured into several components that represent the internal features of the generated MCQ.

Each generated question contains:

Feature	Description
Question Text	The main question derived from the educational content
Option A	First answer option
Option B	Second answer option
Option C	Third answer option
Option D	Fourth answer option
Correct Answer	The correct option among the four choices
Explanation	Short explanation describing the correct answer

These components form the structured representation of each generated question.

Semantic Embeddings

To enable duplicate detection and semantic similarity analysis, the QuizGenix system converts each generated question into a **vector embedding** using a **sentence-transformer model**.

An embedding is a numerical representation of the meaning of a sentence. Instead of comparing text based on exact words, embeddings allow the system to compare **semantic meaning**. This means two questions with similar intent but different wording can still be identified as similar.

Each embedding is represented as a high-dimensional vector, where the position of the vector captures the conceptual meaning of the sentence. By converting questions into embeddings, the system can perform efficient similarity comparisons.

This approach is essential for:

- Identifying duplicate or repeated questions

- Detecting semantically similar questions
- Improving the diversity and quality of generated quizzes

4.2 New Features Created

Feature engineering also involves creating new features derived from the original data. These new features help improve the system's ability to analyze and process information. In QuizGenix, several derived features are created during the processing pipeline.

Overlapping Text Chunks

Instead of dividing the document into completely independent segments, the system creates **overlapping text chunks**.

For example:

Chunk	Content
Chunk 1	Paragraphs 1–3
Chunk 2	Paragraphs 3–5
Chunk 3	Paragraphs 5–7

This overlap ensures that important contextual information appearing at the boundaries of segments is preserved.

Overlapping chunks help the Gemini model maintain contextual continuity when generating questions.

Question Embedding Vectors

For each generated question, the system creates a **semantic embedding vector** using the `sentence-transformer` model. These vectors represent the meaning of the question in a high-

dimensional semantic space. The embedding vectors are used to calculate similarity between questions.

This derived feature allows the system to detect: duplicate questions highly similar questions semantically redundant questions.

Question Similarity Scores

After generating embeddings, the system computes **cosine similarity scores** between pairs of questions.

Cosine similarity measures how closely two embedding vectors align in semantic space. The similarity score is calculated as:

Similarity Score = cosine(embedding₁, embedding₂)

If the similarity score exceeds a predefined threshold (for example 0.85), the questions are considered too similar.

This derived feature helps maintain diversity within the generated quizzes.

Performance Records

The system also generates features related to student performance.

Each quiz attempt produces performance records containing:

Feature	Description
Student ID	Identifier of the student
Question ID	Identifier of the question
Selected Answer	Student's chosen option
Correct Answer	Stored correct option
Result	Correct or Incorrect
Time Taken	Time required to answer

These records are used to identify weak areas and generate personalized quizzes.

4.3 Feature Scaling / Normalization

Feature scaling is an important step in many machine learning systems, where numerical values are transformed into a consistent range. Techniques such as normalization and standardization are commonly used to ensure that features with larger values do not dominate the learning process. This helps improve model performance and stability, especially in algorithms that depend on distance calculations.

However, in the QuizGenix system, the primary data being processed is textual rather than numerical. Since the system focuses on extracting and analyzing text from educational documents, traditional feature scaling methods used for numerical datasets are not directly applicable.

Instead of working with raw numerical features, the system converts text into semantic embeddings, which are vector representations of meaning. These embeddings capture the contextual and semantic relationships between words and sentences.

Although traditional scaling is not required, a form of normalization is implicitly applied during semantic similarity computation. When comparing two question embeddings, the system uses cosine similarity, which measures the angle between two vectors rather than their magnitude.

This approach effectively normalizes the vectors by focusing on their direction, ensuring that:

- Differences in vector magnitude do not affect similarity results
- Semantic meaning is prioritized over numerical size
- Fair and consistent comparison between questions is achieved

As a result, cosine similarity acts as a normalization technique in the embedding space, allowing the system to accurately identify similar or duplicate questions.

Embedding Normalization

Sentence-transformer embeddings are often normalized before similarity calculations. Normalization ensures that the magnitude of embedding vectors does not influence the similarity computation.

Instead of comparing raw vector values, the system computes cosine similarity between normalized vectors.

This approach focuses on the **direction of the vectors**, which represents semantic meaning.

The cosine similarity value ranges between -1 and 1:

Value Meaning

1 Identical meaning

0 No semantic relation

-1 Opposite meaning

In practice, the similarity threshold used by QuizGenix typically ranges between **0.80 and 0.90** to detect duplicate questions.

Text Normalization

Another form of normalization applied in the system is text normalization, which plays a crucial role in preparing the extracted content for accurate processing and analysis. It ensures that variations in text representation do not affect downstream tasks such as duplicate detection and question generation. By converting text into a consistent and standardized format, the system minimizes inconsistencies caused by case differences, spacing issues, or irregular formatting.

Furthermore, text normalization enhances the efficiency of semantic comparison techniques by ensuring that similar content is represented uniformly, making it easier to detect duplicates or near-duplicate questions. It also improves the overall quality of input provided to the **Gemini LLM**, enabling better understanding of the context and resulting in more accurate, clear, and meaningful question generation.

Text normalization includes:

- converting text to lowercase to eliminate case-based inconsistencies
- removing excessive whitespace to maintain uniform spacing
- trimming unnecessary punctuation and special characters
- standardizing formatting to ensure a consistent text structure

These steps ensure that textual comparisons are consistent and reliable when performing duplicate detection using embedding techniques by eliminating superficial differences such as case variations,

Chapter 5

Model Development

Model development is a crucial stage in Artificial Intelligence systems, where appropriate algorithms or computational models are selected to process input data and generate meaningful outputs. In traditional machine learning projects, this stage typically involves collecting labeled datasets, designing model architectures, training models through iterative learning, and optimizing parameters to improve performance.

However, the **QuizGenix system adopts a modern and efficient approach** by utilizing a **pretrained Large Language Model (LLM)** instead of building and training a custom model from scratch. The system integrates the **Gemini 1.5 Flash model** through the Gemini API, with orchestration handled using **LangChain**.

This approach offers several advantages:

- Eliminates the need for large training datasets
- Avoids complex model training and tuning
- Reduces development time and computational cost
- Provides access to state-of-the-art language understanding capabilities

The Gemini model has already been trained on vast amounts of textual data and is capable of performing advanced Natural Language Processing (NLP) tasks such as:

- Understanding context and semantics
- Extracting key concepts from text
- Generating structured outputs like questions, answers, and explanations

In QuizGenix, these capabilities are leveraged to automatically generate **multiple-choice questions (MCQs)** from educational materials such as PDFs.

Model Development Process

Instead of traditional training, the model development in QuizGenix focuses on designing an **AI-driven processing pipeline** that interacts effectively with the Gemini model.

The key steps in this process include:

i. **Text Extraction:**

Educational content is extracted from uploaded PDF documents using text processing libraries.

ii. **Text Preprocessing and Chunking:**

The extracted text is cleaned and divided into smaller chunks to improve model understanding and ensure better context handling.

iii. **Prompt Engineering:**

Carefully designed prompts are created to guide the Gemini model in generating high-quality MCQs. These prompts specify:

- a. The type of questions (e.g., multiple-choice)
- b. Number of questions
- c. Format of output (question, options, correct answer)

iv. **LLM Interaction via LangChain:**

LangChain is used to manage communication with the Gemini API, ensuring smooth and structured input-output handling.

v. **Question Generation:**

The Gemini model processes the prompts and generates context-aware questions based on the input text.

vi. **Post-processing:**

The generated questions are refined, formatted, and filtered to remove duplicates and ensure quality.

5.1 Algorithms Considered

During the development of QuizGenix, several possible approaches for automatic question generation were considered.

Rule-Based Question Generation

One of the earliest approaches for automatic question generation involves rule-based systems. These systems rely on predefined linguistic rules and templates to transform declarative sentences into questions.

For example, a rule-based system might convert a sentence such as:

"Photosynthesis is the process by which plants produce energy." into a question such as:

"What is the process by which plants produce energy?"

Although rule-based systems provide predictable outputs, they have several limitations:

- They require extensive manual rule creation.
- They struggle with complex sentence structures.
- They lack flexibility across different subjects.

Because of these limitations, rule-based methods were not suitable for the QuizGenix system.

Machine Learning-Based Question Generation

Another possible approach involves training machine learning models using labeled datasets containing question-answer pairs. Models such as sequence-to-sequence neural networks can learn to generate questions from textual input.

However, this approach requires:

- Large annotated datasets.
- Extensive model training.
- Computational resources for training and evaluation.

Since the goal of QuizGenix was to build a practical system capable of working with diverse educational materials, collecting and training such datasets was not feasible.

Large Language Model-Based Generation

The final approach adopted in QuizGenix uses a **Large Language Model (LLM)** for question generation. Large Language Models are deep neural networks trained on massive collections of text data. These models learn complex language patterns and can perform a wide variety of natural language processing tasks, including:

- text summarization
- translation
- content generation
- question answering

The **Gemini LLM** provides strong capabilities for understanding context and generating structured responses. By designing appropriate prompts, the system can instruct the model to generate multiple-choice questions based on educational text.

This approach offers several advantages:

- No training dataset required
- High flexibility across subjects
- Ability to generate explanations along with answers
- Reduced development complexity

For these reasons, the QuizGenix system adopts an **LLM-based generation approach**.

5.2 Model Selection

The model selected for the QuizGenix system is **Gemini 1.5 Flash**, accessed through the Gemini API. Gemini is a family of advanced multimodal large language models developed by Google. These models are capable of understanding and generating natural language across a wide range of tasks.

The **Gemini 1.5 Flash** model was selected because it provides:

- Fast response times

- Efficient processing of text inputs
- High-quality language generation
- Support for structured prompt responses

Another important factor in model selection was the availability of the **Gemini API**, which allows developers to integrate the model directly into applications.

Within the QuizGenix system, the Gemini model is accessed through the **LangChain framework**, which simplifies interactions with large language models by providing structured pipelines and prompt templates. The model receives processed educational text as input and generates formatted multiple-choice questions as output.

5.3 Hyperparameter Tuning

In traditional machine learning models, hyperparameter tuning involves adjusting parameters such as learning rate, batch size, or number of training epochs to improve model performance. However, since QuizGenix uses a **pretrained LLM through an API**, the system does not perform direct hyperparameter tuning of the model itself.

Instead, the system focuses on **prompt configuration and generation parameters**, which influence the quality of the generated questions.

The key configurable parameters include:

Number of Questions

The prompt specifies how many questions should be generated from the given text chunk.

Example parameter:

Number of questions per chunk = 5

Output Format Constraints

The prompt instructs the model to generate questions in a specific format containing:

- question text
- four answer options

- correct answer
- explanation

Temperature and Response Control

Although the Gemini API supports parameters that influence response creativity, QuizGenix primarily focuses on maintaining clear and structured outputs. Therefore, prompts are designed to produce deterministic and educationally relevant questions.

Instead of tuning internal neural network parameters, the system relies on **prompt engineering** to guide the model's behavior.

5.4 Training Process

Since the QuizGenix system uses a pretrained Gemini model through an API, no additional training of the model is performed. The system relies entirely on the knowledge already embedded within the pretrained model.

The development process therefore focuses on designing the workflow that prepares input data and processes model outputs.

The main stages in the question generation pipeline are:

- i. Text extraction from PDFdocuments
- ii. Text cleaning and preprocessin
- iii. Text chunk creation
- iv. Prompt construction
- v. LLM questiongeneration
- vi. Response parsing
- vii. Duplicate filtering
- viii. Quiz creation

Each of these stages contributes to transforming raw educational content into structured quiz questions.

5.4.1 Train/Test Split

In machine learning projects, train/test splits are used to divide datasets into training and evaluation sets. However, this concept does not apply to the QuizGenix system.

Because the Gemini LLM is already pretrained, the system does not require training data or evaluation datasets. Instead, the model generates questions dynamically based on user-provided educational content.

Therefore, no train/test split is performed in the system.

5.4.2 Cross-Validation Details

Cross-validation is a model evaluation technique used to estimate how well a trained model will perform on unseen data.

Since the QuizGenix system does not involve training a machine learning model, cross-validation is not required. The system simply uses the pretrained Gemini model to generate questions based on the provided educational text.

Evaluation of the system focuses on the **quality and diversity of generated questions**, which is handled through semantic similarity filtering and instructor review.

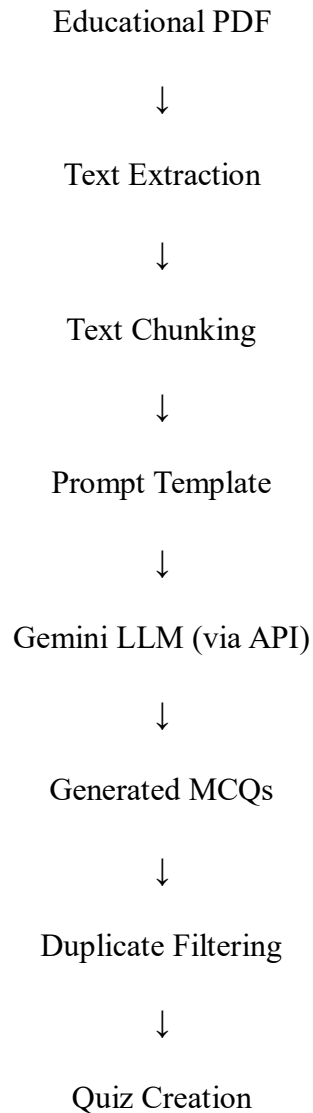
Model Architecture Overview

Although the internal architecture of the **Gemini** model is not directly accessible, it is widely understood that modern large language models are built upon transformer-based neural network architectures. These architectures utilize mechanisms such as self-attention to capture relationships between words and phrases across an entire sequence of text, enabling deep contextual understanding. This allows the model to process input bidirectionally, identify key concepts, and generate coherent and contextually relevant outputs. As a result, the system can effectively transform educational content into meaningful multiple-choice questions, even when dealing with complex or domain-specific material.

Furthermore, transformer-based models are trained on large-scale datasets, allowing them to generalize across a wide range of topics and linguistic patterns. This enhances their ability to interpret diverse educational content and produce accurate, well-structured questions. Their contextual awareness also enables the generation of plausible distractors (incorrect options), which is essential for creating high-quality MCQs that effectively assess a learner's understanding.

In addition, these models can adapt to different prompt structures and instructions, making them highly flexible for tasks such as question generation, answer validation, and explanation synthesis. Their ability to maintain contextual continuity across text segments also ensures consistency in generated outputs, even when processing large documents in chunks.

The overall architecture of the QuizGenix system can be summarized as:



This pipeline enables the system to automatically convert educational materials into quizzes.

Algorithm Complexity Considerations

The computational complexity of the system depends on several components

Text Extraction Complexity

If a document contains n pages, the text extraction process has approximately:

$O(n)$ time complexity because each page must be processed individually.

Question Generation Complexity:

If the document is divided into k text chunks, the system performs k LLM requests, leading to a time complexity of $O(k)$. Each chunk generates multiple questions, so the total number of questions produced is proportional to the number of chunks. This linear complexity ensures scalability, as larger documents can be processed efficiently by handling chunks independently. Additionally, chunk-level processing allows parallel execution of requests, which can significantly reduce overall processing time in distributed environments.

Moreover, this approach improves fault tolerance, as failures in processing one chunk do not affect others. It also enables better resource utilization by distributing workload across multiple processing units. Overall, the design supports efficient, scalable, and reliable question generation for large-scale educational content.

Duplicate Filtering Complexity:

If m questions are generated, the system performs embedding-based similarity comparisons to detect duplicates. A naive pairwise comparison results in approximately $O(m^2)$ similarity checks. However, since the number of questions per quiz is typically limited, this quadratic complexity remains manageable in practice. Furthermore, optimization techniques such as approximate nearest neighbor (ANN) search, clustering, or threshold-based filtering can be applied to reduce computational overhead. These methods improve efficiency while maintaining the quality and diversity of the final question set, ensuring that only unique and meaningful questions are presented to the learner.

Chapter 6

Model Evaluation

Model evaluation is an essential stage in Artificial Intelligence systems to assess how effectively the system performs its intended tasks. In traditional machine learning projects, model evaluation typically involves testing the trained model on unseen data using statistical performance metrics such as accuracy, precision, recall, and F1-score.

However, the QuizGenix system differs from traditional machine learning systems because it does not train a predictive model. Instead, it uses a **pretrained large language model (Gemini 1.5 Flash)** to generate multiple-choice questions from educational text. Therefore, the evaluation process focuses on measuring the **quality, diversity, and usefulness of the generated questions**, as well as the system's ability to support quiz delivery and performance tracking. The evaluation of QuizGenix considers several aspects of system performance, including:

- Quality and clarity of generated MCQs
- Diversity of questions after duplicate filtering
- System responsiveness during quiz generation
- Accuracy of quiz evaluation and scoring
- Effectiveness of personalized quizzes

These factors help determine whether the system successfully supports automated assessment generation.

6.1 Evaluation Metrics

Since the system generates educational questions rather than numeric predictions, evaluation metrics are based primarily on **qualitative assessment criteria** and system performance indicators.

The following metrics are used to evaluate the QuizGenix system.

Relevance

Relevance measures how well the generated question reflects the information contained in the original educational text. A relevant question should accurately represent the concepts described in the source material.

For example, if the uploaded document discusses machine learning algorithms, the generated questions should focus on definitions, concepts, and applications related to machine learning.

Questions that reference unrelated concepts are considered irrelevant.

Clarity

Clarity measures how understandable and grammatically correct the generated questions are. A clear question should:

- contain proper grammar

- use understandable wording

- present answer options that are clearly distinguishable

The Gemini LLM generally produces well-structured sentences, which improves the clarity of generated questions.

Diversity

Diversity measures the variety of questions within a quiz. A diverse quiz covers multiple topics or concepts from the educational text rather than repeating similar questions.

Semantic similarity filtering plays an important role in improving question diversity.

Correct Answer Accuracy

Each generated question includes a correct answer identified by the language model. The evaluation process verifies whether the correct answer is logically consistent with the source text. Questions with incorrect answers or misleading explanations are considered lower quality. Additionally, such low-quality questions are filtered out to ensure only accurate and reliable content is presented to the learner.

System Response Time

System responsiveness is also an important metric. The time required to generate quizzes depends on factors such as:

- size of the uploaded document
- number of text chunks

6.2 Performance Results

The QuizGenix system was tested using several educational PDF documents containing lecture notes and study materials. These documents were processed through the full pipeline, including text extraction, chunking, question generation, and duplicate filtering.

The results demonstrated that the Gemini LLM can successfully generate multiple-choice questions from a wide variety of educational content.

A typical quiz generation workflow produced the following results:

Stage	Output
PDF Upload	Educational document provided by user
Text Extraction	Text successfully extracted from document
Text Chunking	Document divided into multiple chunks
Question Generation	Multiple MCQs generated per chunk
Duplicate	Filtering Similar questions removed
Final Quiz	Diverse set of questions created

The majority of generated questions were found to be grammatically correct and relevant to the source material.

The duplicate filtering mechanism effectively removed redundant questions that referred to similar concepts.

Overall, the system demonstrated consistent performance in generating quizzes across different educational topics.

The system maintained consistent question formatting across different quizzes and subjects.

Generated answer options were clear, logically structured, and aligned with the source material. The questions reflected key concepts accurately while showing natural variation in difficulty levels. The filtering and chunk-based processing ensured contextual relevance and minimized semantic repetition. Overall quiz quality improved through iterative testing, prompt tuning, and efficient preprocessing steps.

6.3 Model Comparison

Although the QuizGenix system uses the Gemini LLM for question generation, it is useful to compare this approach with alternative question generation methods. The following table summarizes the differences between various approaches.

Method	Advantages	Limitations
Rule-Based Question Generation	Simple implementation	Limited flexibility
Machine Learning-Based Systems	Can learn from training data	Requires large datasets
Neural Sequence Models	More advanced language understanding	Requires training and optimization
LLM-Based Generation (Gemini)	High flexibility, strong language understanding	Requires API access
QuizGenix System	Combines LLM generation with filtering and quiz management	Depends on LLM availability

Compared with traditional approaches, the Gemini-based generation method provides several benefits.

First, it eliminates the need for training datasets and complex model training procedures. Second, it allows the system to generate questions from any educational text without requiring domainspecific training.

Additionally, the integration of semantic similarity filtering improves the quality and diversity of the generated quizzes.

6.4 Error Analysis

Although the QuizGenix system performs effectively in most situations, some limitations and potential errors were observed during testing.

Repetitive Questions

In some cases, the Gemini model may generate questions that refer to similar concepts using slightly different wording. Without filtering, this could lead to repetitive quizzes.

The duplicate filtering mechanism using sentence-transformer embeddings helps reduce this problem by identifying semantically similar questions.

Simplistic Questions

Some generated questions focus primarily on basic factual recall rather than deeper conceptual reasoning. This occurs because the language model tends to extract definitions or simple statements from the source text.

While these questions are still useful for learning, more advanced reasoning questions may require additional prompt engineering.

Incorrect Answer Selection

In rare cases, the model may assign an incorrect answer option due to ambiguous wording or complex sentences in the source material.

These situations highlight the importance of instructor review when generating quizzes for formal assessments.

Chapter 7

Deployment Strategy

Deployment is the process of making a developed system available for use by end users. After the development and testing phases are completed, the application must be deployed in an environment where users can interact with the system through its interface.

In the QuizGenix project, deployment focuses on running the AI-powered quiz generation system as a

Django web application on a local server. The system integrates multiple components, including the Django framework, Gemini API, LangChain pipeline, and SQLite database. These components work together to provide a functional platform where teachers and students can interact with the automated quiz generation system.

The deployment strategy ensures that the system operates smoothly and that all components communicate effectively with each other. The deployment architecture includes the web interface, backend processing modules, AI services, and the database used for storing quiz and user data.

The overall deployment workflow includes installing required software dependencies, configuring API keys, setting up the Django application, and running the development server to make the system accessible through a web browser.

7.1 Model Serialization

Model serialization refers to the process of saving a trained machine learning model so that it can be reused later without retraining. In many machine learning systems, models are serialized into files such as .pkl, .joblib, or .h5 formats.

However, the QuizGenix system does not train its own machine learning model. Instead, it uses the **pretrained Gemini 1.5 Flash large language model**, which is accessed through the Gemini API. Since the model is hosted externally by Google, the system does not need to store or serialize the model locally.

Instead of model serialization, the QuizGenix system stores the following data within the local database:

- generated quiz questions
- quiz metadata

- student accounts
- quiz attempts
- performance records

The Gemini model remains accessible through API calls whenever the system needs to generate new questions.

Therefore, the concept of model serialization is replaced with **API-based model access**, where the external model is invoked dynamically when needed.

7.2 Deployment Tools / Frameworks

The QuizGenix system uses several software tools and frameworks that support the deployment and execution of the application.

Django Web Framework

The core of the application is built using the Django framework. Django provides a robust environment for developing web applications and includes features such as:

- URL routing
- database interaction through the Object Relational Mapper (ORM)
- user authentication and session management
- template-based user interface rendering

Django also supports rapid development and provides a secure architecture for handling user data.

Python Programming Language

Python serves as the primary programming language for implementing the system's backend logic. Python is widely used in artificial intelligence applications due to its extensive ecosystem of libraries and frameworks.

LangChain Framework

LangChain acts as an interface between the Django application and the Gemini large language model. It simplifies the process of interacting with LLMs by providing tools for prompt templates, structured pipelines, and response parsing.

LangChain allows the system to send educational text to the Gemini model and retrieve generated multiplechoice questions in a structured format.

Gemini API

The Gemini API provides access to the Gemini 1.5 Flash large language model. The API receives text input and generates structured textual responses based on the prompt instructions. The QuizGenix system uses the Gemini API to perform AI-based question generation.

SQLite Database

The application uses SQLite as its database system. SQLite is a lightweight relational database that is well suited for local applications and development environments.

The database stores all application data, including:

- user accounts
- quizzes
- questions
- quiz attempts
- performance records

PDF Processing Libraries

The system uses Python libraries such as **pdfplumber** or **pypdf** to extract text from uploaded PDF documents. These libraries allow the application to process educational documents and convert them into textual input for the AI model. **Excel Processing Libraries**

The system uses **openpyxl** to read Excel files uploaded by teachers. These Excel files contain student information, which the system uses to create student accounts automatically.

7.3 Integration Process

The deployment of QuizGenix involves integrating multiple components into a single functioning system.

Each component performs a specific role in the overall quiz generation workflow.

The integration process follows several steps.

Step 1: User Access Through Web Interface

Users access the system through a web browser. Django renders the user interface using HTML templates, allowing users to log in, upload documents, create quizzes, and attempt assessments.

Step 2: Document Upload and Storage

When a teacher uploads a PDF document, the file is stored in the application's media directory.

The system then processes the file using the PDF extraction module.

Step 3: Text Extraction

The PDF extraction module reads the uploaded document and extracts textual content from each page. The extracted text is then cleaned and prepared for processing.

Step 4: Text Chunking

The cleaned text is divided into smaller segments called text chunks. These chunks allow the system to send manageable input sizes to the Gemini model.

Step 5: AI Question Generation

Each text chunk is processed by the Gemini model through the LangChain interface. The prompt template instructs the model to generate multiple-choice questions with answer options and explanations.

Step 6: Duplicate Filtering

The generated questions are analyzed using sentence-transformer embeddings to detect duplicate or highly similar questions. Redundant questions are removed to improve quiz diversity.

Step 7: Quiz Creation

The filtered questions are stored in the database and associated with a specific quiz created by the instructor.

Step 8: Student Quiz Attempt

Students log in to the system and attempt quizzes through the web interface. Timer functionality may restrict the duration of the quiz.

Step 9: Automatic Evaluation

When a student submits a quiz, the system compares the selected answers with the stored correct answers and calculates the final score.

Step 10: Performance Tracking

The system records performance data for each student, including correct and incorrect answers. This information is used to generate personalized quizzes that focus on weak areas.

7.4 Output screenshots:

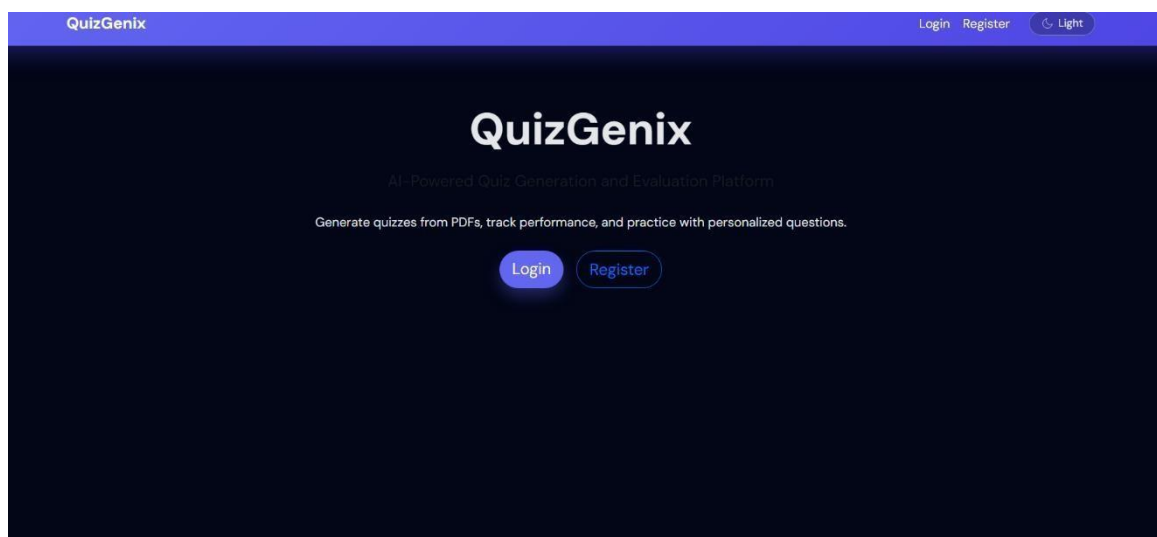


Fig 7.4.1: Home page of QuizGenix displaying the AI-powered quiz generation platform. The interface provides login and registration options along with dark/light mode support. It serves as the entry point for teachers and students to access the system.

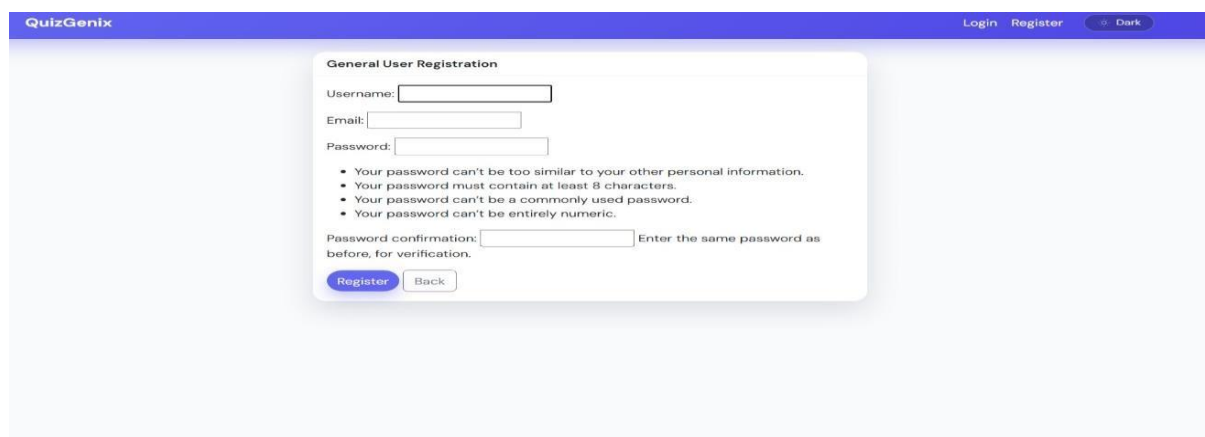
The screenshot shows the 'General User Registration' form on the QuizGenix website. The form is centered on a light blue background. It includes input fields for 'Username', 'Email', and 'Password'. Below the password field, there are four bullet points listing password requirements: it can't be too similar to personal information, must be at least 8 characters, can't be a commonly used password, and can't be entirely numeric. There is also a 'Password confirmation' field with a note to 'Enter the same password as before, for verification.' At the bottom of the form are two buttons: 'Register' (in blue) and 'Back' (in white). The top of the page has a dark blue header with 'QuizGenix' on the left and 'Login Register Dark' on the right.

Fig 7.4.2: This screen shows the registration form for general users. Users need to enter details like username, email, and password to create an account. The form also includes password rules to ensure security. After filling the details, users can click the Register button to create their account.

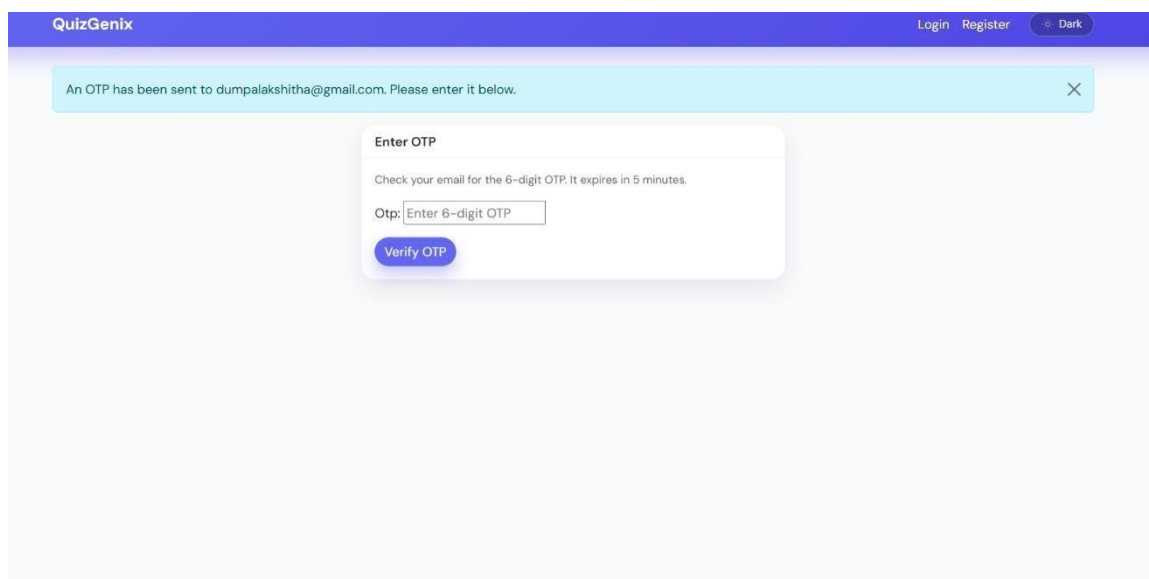
The screenshot shows the OTP verification screen on the QuizGenix website. At the top, a light blue notification bar states: 'An OTP has been sent to dumpalakshitha@gmail.com. Please enter it below.' Below this is a white box titled 'Enter OTP'. Inside the box, it says 'Check your email for the 6-digit OTP. It expires in 5 minutes.' There is an input field labeled 'Otp:' with the placeholder text 'Enter 6-digit OTP'. At the bottom of the box is a blue button labeled 'Verify OTP'. The top of the page has a dark blue header with 'QuizGenix' on the left and 'Login Register Dark' on the right.

Fig 7.4.3: Login page where registered users can securely access the QuizGenix platform. Users enter their username and password to authenticate into the system. The interface is simple and user-friendly for quick access.

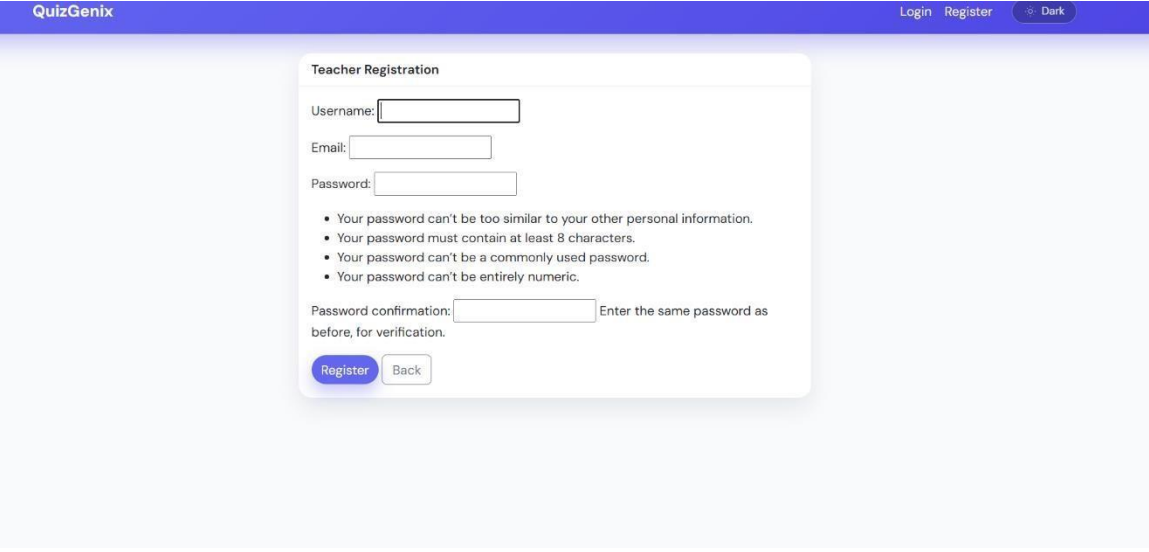
The image shows a web browser window with the QuizGenix logo in the top left. The top navigation bar includes links for 'Login', 'Register', and a 'Dark' theme toggle. The main content area features a 'Teacher Registration' form. The form has input fields for 'Username:', 'Email:', and 'Password:'. Below the password field, there is a list of password requirements: 'Your password can't be too similar to your other personal information.', 'Your password must contain at least 8 characters.', 'Your password can't be a commonly used password.', and 'Your password can't be entirely numeric.'. A 'Password confirmation:' field is also present, with a prompt to 'Enter the same password as before, for verification.'. At the bottom of the form are two buttons: 'Register' and 'Back'.

Fig 7.4.4: Teacher registration page used for creating teacher accounts in the system. It includes authentication details and password validation features for secure registration. Teachers can use this account to create and manage quizzes.

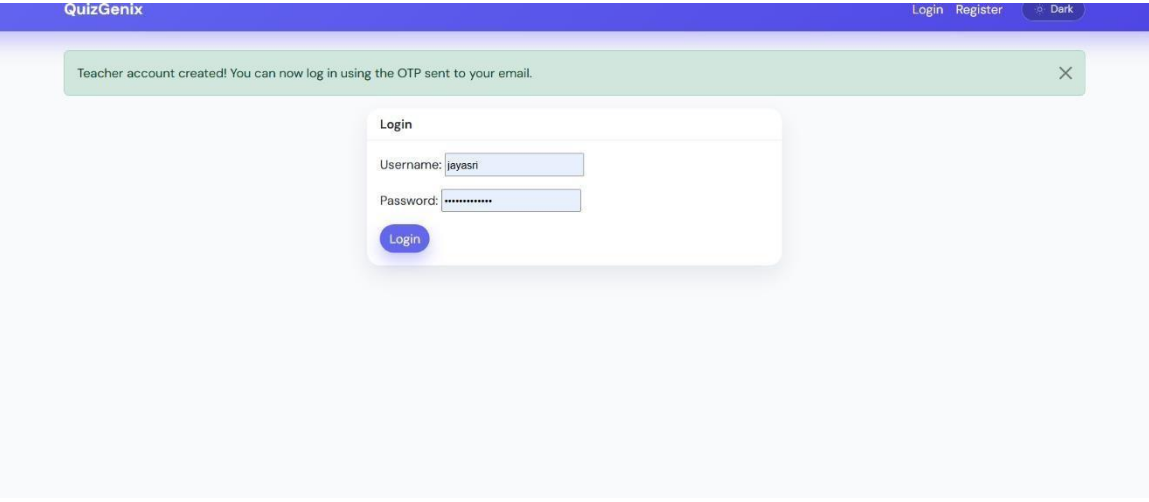
The image shows a web browser window with the QuizGenix logo in the top left. The top navigation bar includes links for 'Login', 'Register', and a 'Dark' theme toggle. A green success message banner at the top reads: 'Teacher account created! You can now log in using the OTP sent to your email.' Below this, there is a 'Login' form. The form has input fields for 'Username:' (containing the text 'jayasn') and 'Password:' (displayed as masked characters). A 'Login' button is located at the bottom of the form.

Fig 7.4.5: Teacher login confirmation page displaying successful account creation and login instructions. The page confirms that the teacher account has been registered successfully. Users can then proceed to access the dashboard.

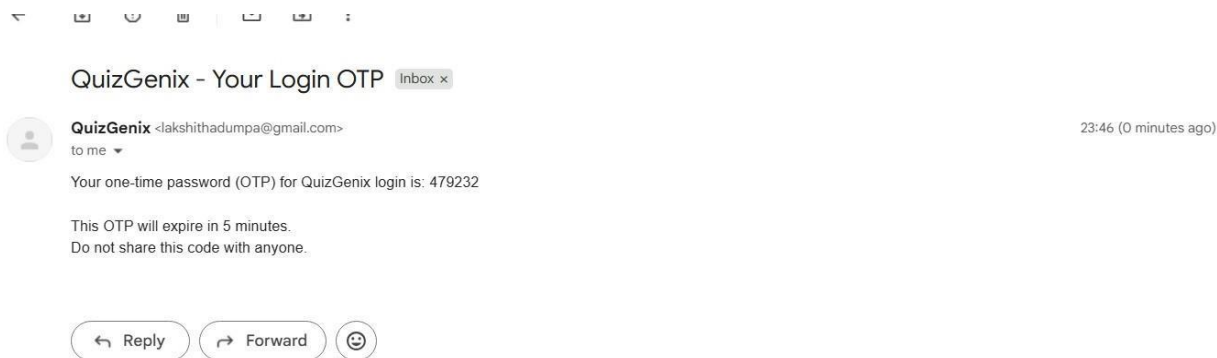


Fig 7.4.6: OTP verification page requesting users to enter the one-time password sent to their email. This step enhances login security using two-factor authentication. Users must verify the OTP within the given time limit

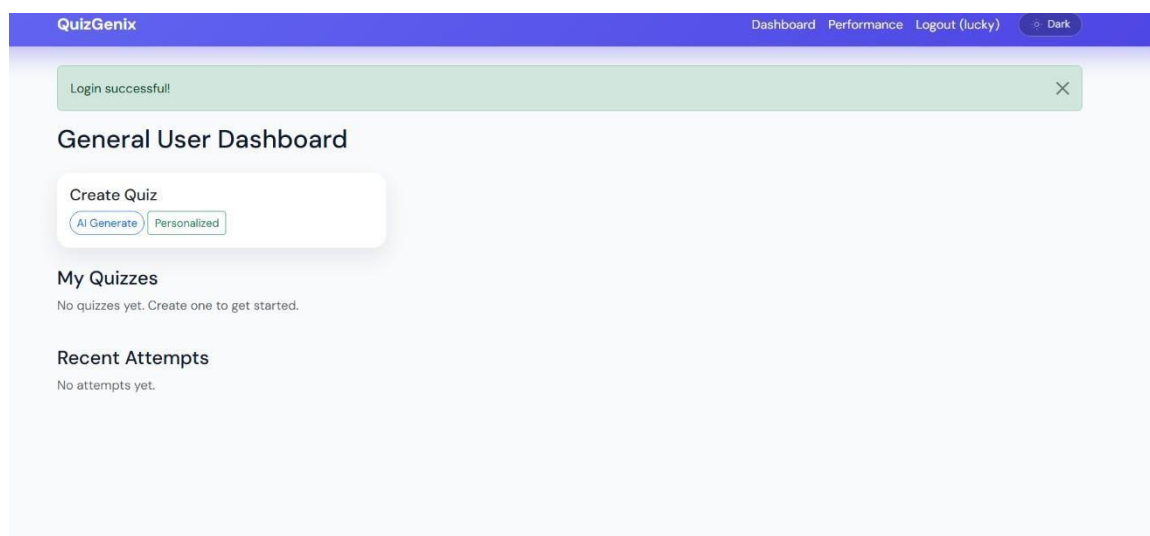


Fig 7.4.7: General user dashboard showing quiz creation options, available quizzes, and recent attempts. Students can practice quizzes and track their performance from this page. It provides a personalized learning environment.

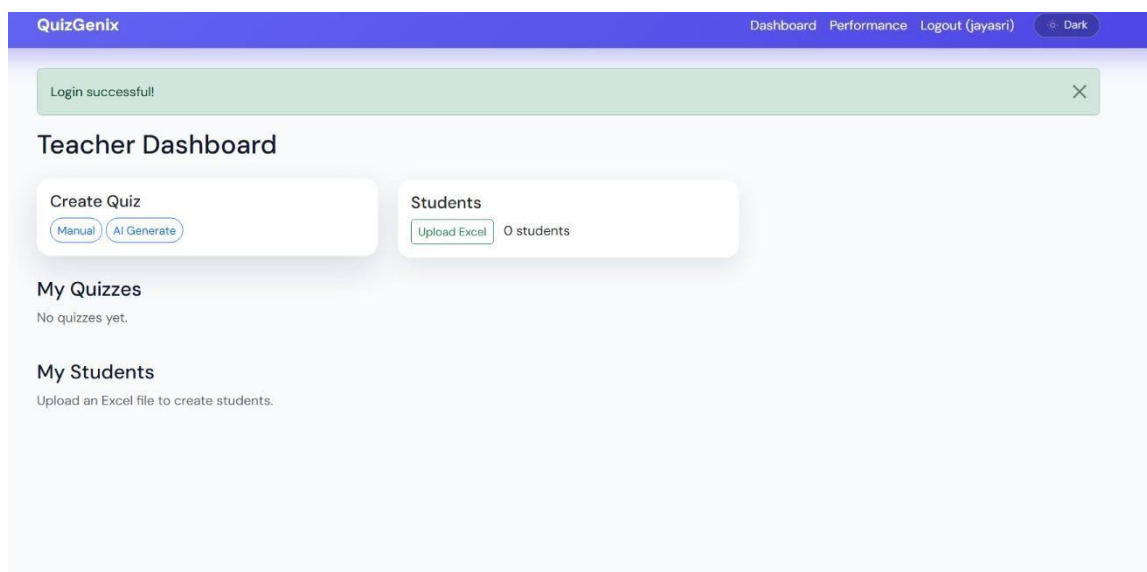


Fig 7.4.8: Teacher dashboard showing quiz creation and student management functionalities. Teachers can create quizzes manually or using AI-generated content. The dashboard also provides access to uploaded student information.

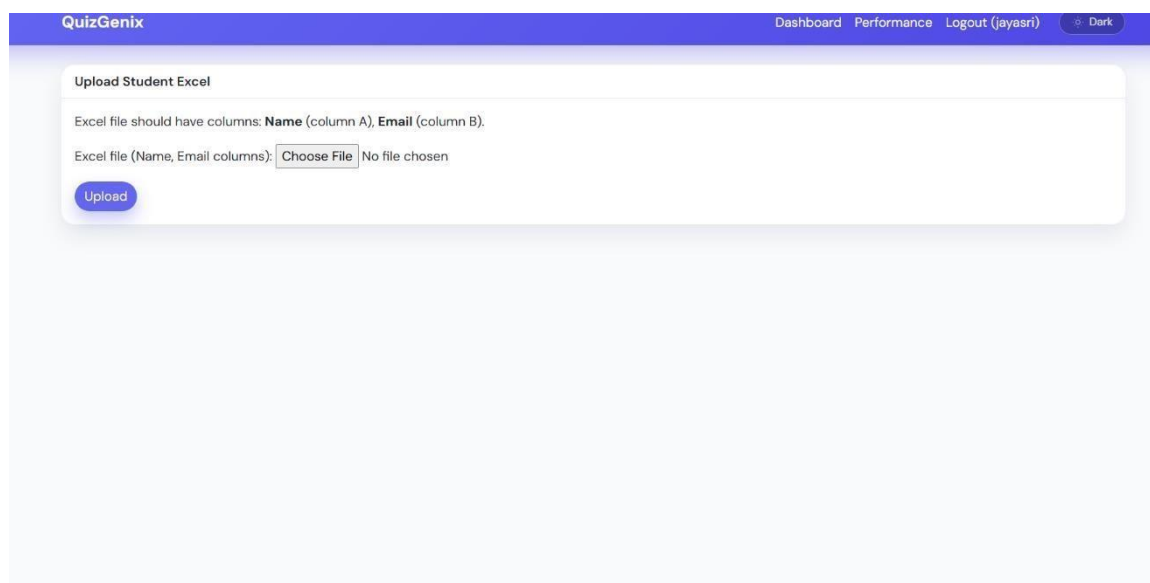


Fig 7.4.9: Student upload page enabling teachers to upload Excel files containing student details. The system accepts names and email addresses for bulk student account creation. This feature simplifies classroom management.

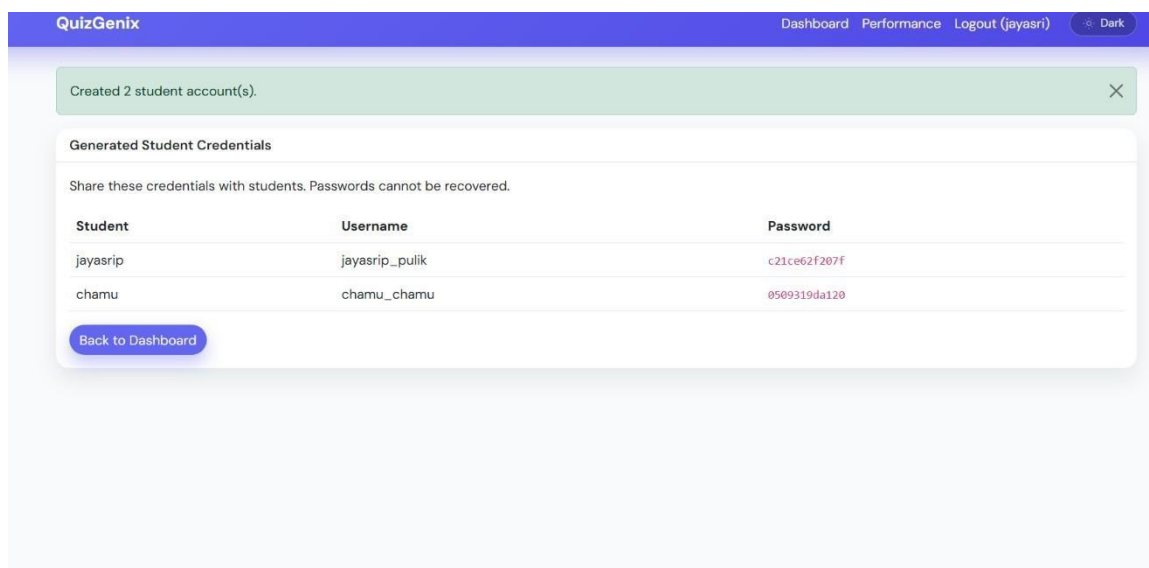


Fig 7.4.10: Generated student credentials page displaying usernames and passwords for created student accounts. Teachers can share these credentials with students for login access. It automates student account generation efficiently.

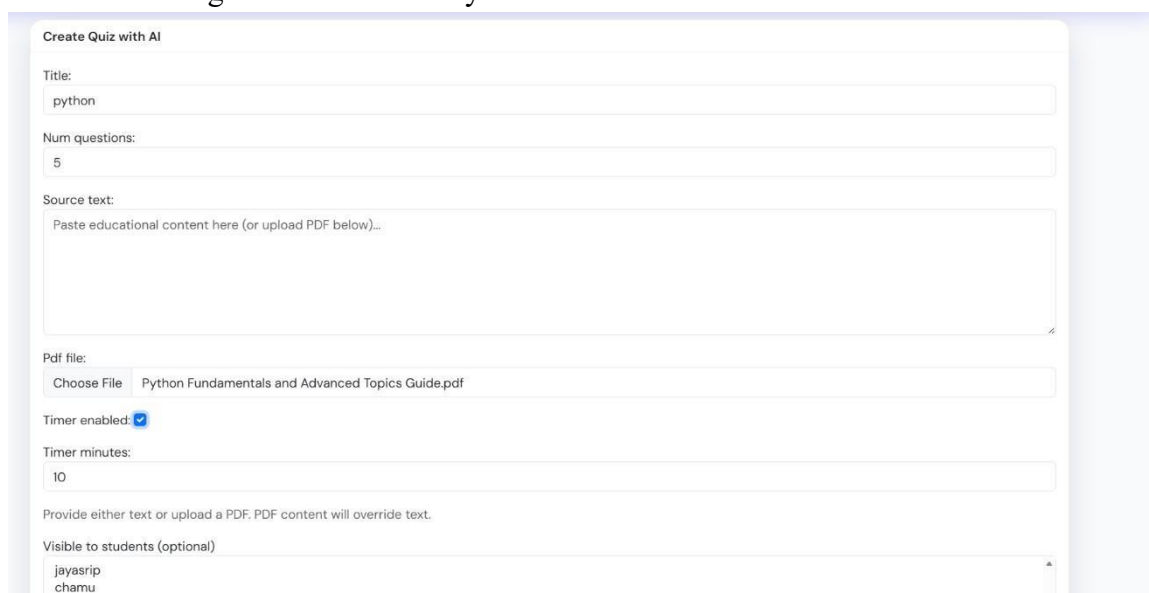


Fig 7.4.11: AI quiz creation page where teachers can generate quizzes using educational text or uploaded PDF files. The page allows selection of the number of questions and timer settings. Teachers can also assign quiz visibility to students.

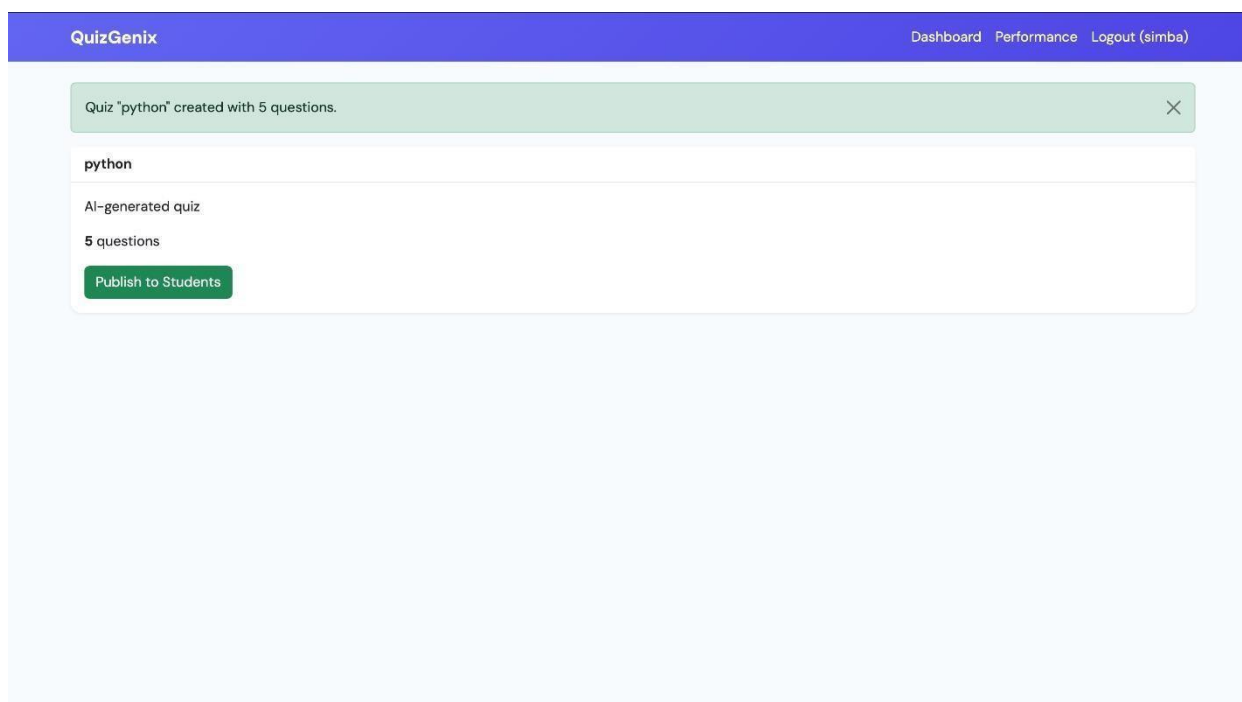


Fig 7.4.12: Quiz creation success page displaying the generated quiz and publishing options. Teachers can review the AI-generated quiz before making it available to students. This ensures better control over quiz distribution.

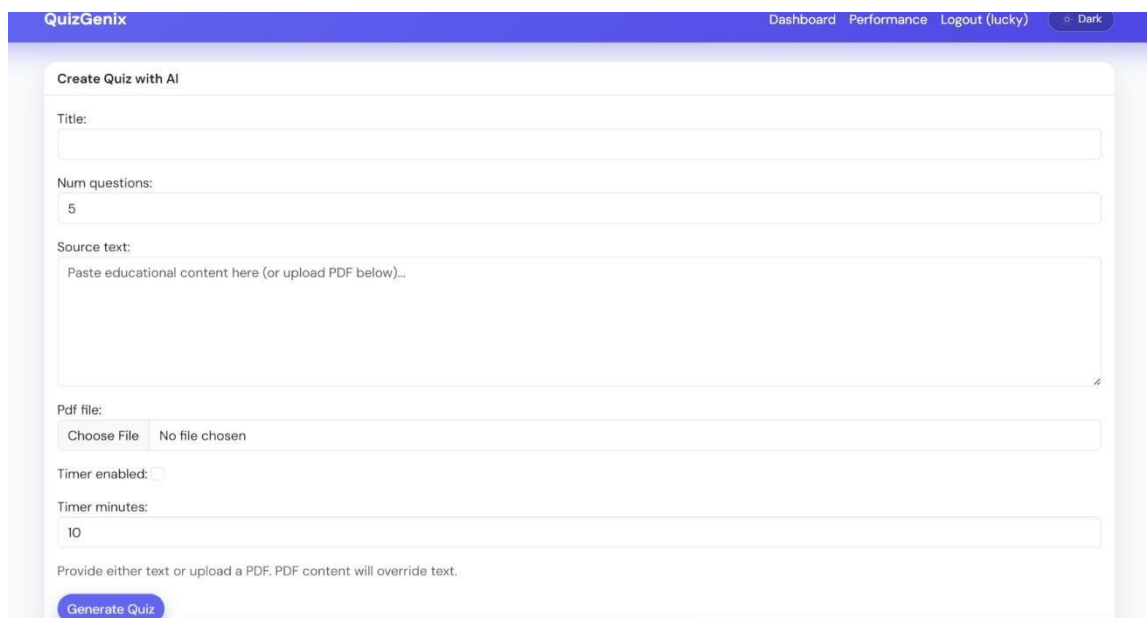


Fig 7.4.13: General user dashboard showing quiz creation options, available quizzes, and recent attempts. Students can practice quizzes and track their performance from this page. It provides a personalized learning environment.

QuizGenix Dashboard Performance Logout (gayatris_gayat)

python

1. According to the "Types of variables" section in the provided content, which three specific categories of variables are discussed in Python OOP?

- ☐ A. Private, Public, and Protected variables
- ☐ B. Instance, Static, and Local variables
- ☐ C. Global, Class, and Method variables
- ☐ D. Dynamic, Constant, and Temporary variables

2. Under the category of "Polymorphism," which of the following sub-topics is explicitly mentioned in the table of contents?

- ☐ A. Exception Handling
- ☐ B. Garbage Collector
- ☐ C. Operator overloading
- ☐ D. Syntactical errors

3. According to the provided content, what are the two specific types of errors discussed under the "Exception Handling" section?

- ☐ A. Logic errors and Logical errors
- ☐ B. Compilation errors and System errors
- ☐ C. Syntactical errors and Runtime errors
- ☐ D. Design errors and Internal errors

4. Which specific keyword is associated with "Generator Functions" in the provided educational syllabus?

- ☐ A. Return keyword
- ☐ B. Yield keyword
- ☐ C. Global keyword
- ☐ D. Lambda keyword

Fig 7.4.14: Quiz attempt interface displaying multiple-choice questions for students to answer online. The system presents questions in an organized format with selectable options. It supports interactive quiz participation and evaluation.

QuizGenix Dashboard Performance Logout (gayatris_gayat)

Quiz Result: python

Thank you for completing the quiz!

Score: 2/5

Time: 716 seconds

1. According to the "Types of variables" section in the provided content, which three specific categories of variables are discussed in Python OOP?

Your answer: D Dynamic, Constant, and Temporary variables

Correct answer: B — Instance, Static, and Local variables

Explanation: The Table of Contents explicitly lists "Instance variables," "Static variables," and "Local variables" under the "Types of variables" heading.

2. Under the category of "Polymorphism," which of the following sub-topics is explicitly mentioned in the table of contents?

Your answer: C Operator overloading

Explanation: The content lists "Operator overloading," "Method overloading," and "Constructor overloading" as specific sub-sections under the main heading of "Polymorphism."

3. According to the provided content, what are the two specific types of errors discussed under the "Exception Handling" section?

Your answer: B Compilation errors and System errors

Fig 7.4.15: This screen shows the quiz result after a student completes the quiz. It displays the score, time taken, and detailed feedback for each question. The system shows the user's answer, the correct answer, and an explanation. This helps students understand their mistakes and improve their learning.

Chapter 8 Monitoring and Maintenance

Monitoring and maintenance are essential components of any software system to ensure reliable operation, long-term usability, and continuous improvement. After deployment, the system must be observed regularly to verify that it functions correctly, handles user interactions effectively, and maintains acceptable performance levels.

In the case of the QuizGenix system, monitoring focuses on observing the behavior of the AI-powered quiz generation pipeline, tracking student performance data, and ensuring that the integration between the Django application and the Gemini large language model operates smoothly.

Since QuizGenix relies on an external AI model accessed through the Gemini API, monitoring also involves ensuring that the API integration remains stable and that responses from the model continue to produce meaningful questions.

Maintenance activities include updating system dependencies, improving prompt templates for better question generation, and maintaining the database containing quizzes, questions, and student performance records.

Effective monitoring and maintenance ensure that the system continues to provide accurate quiz generation and reliable evaluation for students and instructors.

8.1 Performance Tracking

Performance tracking is a key component of the QuizGenix system. The system continuously records data related to quiz attempts and student interactions. This data helps instructors understand how students perform on different quiz questions and identify areas where learners may require additional practice.

Each time a student attempts a quiz, the system records several important pieces of information in the database. These records are stored as **performance records** associated with individual quiz attempts.

The stored information includes:

Data Field Description

Student ID	Identifier of the student attempting the quiz
Quiz ID	Identifier of the quiz being attempted
Question ID	Identifier of the question answered
Selected Answer	Option chosen by the student
Correct Answer	Correct answer stored in the system
Result	Indicates whether the answer is correct or incorrect
Time Taken	Time spent completing the quiz

These performance records serve multiple purposes.

First, they allow instructors to analyze how well students understand the quiz material. If a large number of students answer a specific question incorrectly, this may indicate that the concept requires further explanation.

Second, the stored performance data enables the system to generate **personalized quizzes**. By identifying questions that students previously answered incorrectly, the system can generate new quizzes that focus specifically on those weak areas.

Third, the recorded time taken for quiz attempts allows instructors to evaluate how efficiently students complete assessments. This information may reveal whether quizzes are too easy, too difficult, or require adjustments in the timer settings.

Overall, performance tracking plays a critical role in improving the learning process and supporting adaptive assessment.

8.2 Retraining Strategy

In traditional machine learning systems, retraining involves updating the model using new datasets so that it can improve its predictions over time. However, the QuizGenix system uses a **pretrained Gemini large language model**, which is accessed through the Gemini API.

Because the Gemini model is hosted externally and maintained by its provider, the QuizGenix system does not perform model retraining locally. Instead, improvements in the underlying language model are automatically provided when the API provider updates the model.

Although the system does not retrain the model, improvements can still be achieved through **prompt refinement and system updates**.

The retraining strategy in the context of QuizGenix therefore focuses on the following activities.

Prompt Optimization

The quality of generated questions depends heavily on the structure of the prompt provided to the language model. By refining the prompt template, developers can guide the model to produce more accurate and structured questions. Prompt improvements may include:

- specifying stricter formatting requirements
- requesting more diverse questions
- encouraging conceptual reasoning questions

Improved Text Preprocessing

Enhancing the text extraction and cleaning pipeline can also improve the quality of generated questions.

Better preprocessing reduces noise and ensures that the model receives clear input text.

Duplicate Filtering Enhancements

Adjusting the similarity threshold used in semantic duplicate filtering may improve quiz diversity. A more optimized threshold can remove redundant questions while preserving meaningful variations.

Continuous System Updates

As new versions of language models become available through the Gemini API, the system can be updated to use improved models without requiring major architectural changes.

Thus, although the system does not retrain models locally, continuous improvements are achieved through prompt engineering and system refinement.

8.3 Monitoring Tools

Monitoring tools help developers observe the behavior of the system and identify potential issues that may arise during operation.

In the QuizGenix system, monitoring primarily occurs through application-level logging, database records, and manual review of generated quizzes. **Application Logs**

Django provides built-in logging mechanisms that record system events and errors. These logs help developers detect issues such as:

- API connection failures
- document processing errors
- database operation failures

By reviewing application logs, developers can quickly identify and resolve technical issues.

Database Monitoring

The SQLite database used by the system stores all application data, including quizzes, questions, and performance records. Monitoring database activity ensures that records are stored correctly and that queries execute efficiently.

Regular database checks help ensure that the stored information remains consistent and accurate.

AI Output Monitoring

Since the Gemini model generates questions dynamically, instructors may occasionally review generated quizzes to ensure that the questions are relevant and accurate.

Manual review helps verify that the generated questions align with the educational content and maintain acceptable quality standards.

System Usage Monitoring

The system can also monitor usage patterns, such as:

- number of quizzes generated
- number of quiz attempts
- frequency of personalized quiz creation

Chapter 9

Conclusion

The increasing use of digital learning environments has created a strong demand for efficient and scalable assessment systems. Traditional quiz creation methods require instructors to manually design questions, evaluate responses, and track student performance, which becomes time-consuming in large classrooms and online platforms. QuizGenix addresses these challenges by providing an AI-powered system that automates quiz generation and evaluation from educational materials.

The system processes PDF documents uploaded by instructors by extracting and cleaning textual content, which is then divided into manageable chunks. These segments are sent to the **Gemini 1.5 Flash** via API, where multiple-choice questions (MCQs) are generated along with answer options, correct answers, and explanations. To maintain quality and diversity, semantic duplicate filtering using sentence-transformer embeddings is applied to remove similar or repetitive questions.

QuizGenix also includes features that enhance usability in educational settings. Teachers can upload Excel files to create student accounts automatically, while students can log in, attempt quizzes through a **Django**-based interface, and receive instant evaluation. The system tracks performance data such as accuracy, incorrect responses, and time taken, enabling the generation of personalized quizzes that focus on students' weak areas. This supports adaptive learning and improves concept retention.

The platform integrates multiple technologies, including Python, **LangChain**, Gemini, and sentence-transformer embeddings, to deliver a complete automated assessment solution. While the system generates relevant and grammatically correct questions, some limitations remain, such as a tendency toward factual questions and dependence on the quality of input text.

Despite these limitations, QuizGenix demonstrates the potential of AI in modern education by reducing instructor workload, enabling scalable assessments, and providing personalized learning experiences. It represents a significant step toward integrating intelligent systems into educational environments and improving both teaching and learning outcomes. QuizGenix is an AI-powered platform that automates quiz generation and evaluation from educational PDFs using the **Gemini 1.5 Flash**, reducing manual effort for instructors

Chapter 10

Future Scope of the Project

Although the QuizGenix system provides a functional platform for automated quiz generation and evaluation, several improvements and enhancements can be implemented in the future to extend the capabilities of the system.

One potential improvement is the introduction of additional question types beyond multiple-choice questions. Future versions of the system could support short-answer questions, descriptive questions, and true-or-false questions. These additional formats would allow instructors to evaluate deeper levels of student understanding.

Another important enhancement involves the integration of retrieval-based question generation techniques. Currently, the system generates questions directly from text chunks extracted from educational documents. In the future, the system could incorporate vector databases that store embeddings of document content. This would allow the system to retrieve the most relevant content segments before generating questions, thereby improving the accuracy and contextual alignment of generated quizzes.

The QuizGenix platform could also benefit from advanced analytics features. By analyzing student performance data, the system could generate detailed reports that highlight learning trends, identify difficult concepts, and recommend targeted study materials. Such analytics could help instructors make informed decisions about course design and teaching strategies.

Another potential improvement involves deploying the system on cloud-based infrastructure. Currently, the system operates as a locally deployed Django application. Deploying the system on cloud platforms such as Google Cloud, AWS, or other hosting environments would enable remote access and support larger numbers of users.

Future development could also focus on integrating the system with existing learning management systems (LMS). Integration with platforms such as Moodle or Google Classroom would allow quizzes generated by QuizGenix to be directly incorporated into course management systems used by educational institutions. Another area of potential enhancement involves multilingual support. Currently, the system processes educational text primarily in English. By incorporating multilingual language models, the platform could generate quizzes in multiple languages, making it suitable for a wider range of educational environments.

Chapter 11

Appendix

11.1 Code Repository

The QuizGenix system was implemented using Python and the Django web framework. The source code of the project contains multiple modules responsible for different functionalities of the system.

The main components of the project include:

- Django web application for user interaction
- PDF extraction module for processing educational documents
- AI-based question generation module using Gemini API
- semantic duplicate filtering module using sentence-transformer embeddings
- quiz management system for storing quizzes and questions
- performance tracking system for recording student quiz attempts

The project source code is organized into several Python modules that implement these functionalities. The repository contains the necessary configuration files, Django models, views, templates, and service modules required to run the system.

11.1.1 Github link

<https://github.com/Lakshithadumpa/QuizGenix-An-AI-powered-platform-for-automated-generation-and-evaluation-of-MCQs->

11.1.2 Complete Project Code

ai_generator.py

"""

QuizGenix - AI Quiz Generation (Gemini / OpenAI)

"""

```
import re
from django.conf import settings
```

```
# ----- #
```

```
Gemini SDK (Preferred)
```

```
# ----- try:
```

```
    from google import genai
```

```
    HAS_GEMINI_SDK = True except
```

```
ImportError:
```

```
    HAS_GEMINI_SDK = False
```

```
# -----
```

```
# LangChain Fallback
```

```
# ----- try:
```

```
    from langchain_google_genai import ChatGoogleGenerativeAI
```

```
    from langchain_openai import ChatOpenAI
```

```
    from langchain.schema import HumanMessage
```

```
    HAS_LANGCHAIN = True except
```

```
ImportError:
```

```
    HAS_LANGCHAIN = False
```

```
# -----
```

```
# Prompt Template
```

```
# -----
```

QUIZ_PROMPT = """"You are an expert educational content creator. Generate unique multiplechoice questions (MCQs) from the given educational content.

RULES:

- Generate exactly {num_questions} questions.
- Each question MUST have exactly 4 options (A, B, C, D).
- Include the correct answer (A, B, C, or D).
- Include a short explanation for the correct answer.
- Ensure NO duplicate or semantically similar questions.
- Use clear, educational language.
- Base questions ONLY on the provided content.

OUTPUT FORMAT (use this exact format for each question):

---QUESTION---

Question: [your question here]

Option A: [option text]

Option B: [option text] Option

C: [option text]

Option D: [option text]

Correct Answer: [A/B/C/D]

Explanation: [brief explanation]

---END---

Educational content:

{content}

""""

```
# LLM Fallback Loader
# ----- def
get_llm():
    """Return configured LLM (Gemini or OpenAI) for LangChain fallback."""
    provider = getattr(settings, "LLM_PROVIDER", "gemini")

    if provider == "openai" and getattr(settings, "OPENAI_API_KEY", None):
        return ChatOpenAI(
            model="gpt-4o-mini",
            api_key=settings.OPENAI_API_KEY,
            temperature=0.7,
        )

    if getattr(settings, "GOOGLE_API_KEY", None):
        return ChatGoogleGenerativeAI(
            model=getattr(settings, "GEMINI_MODEL", "gemini-2.0-flash"),
            google_api_key=settings.GOOGLE_API_KEY,
            temperature=0.7,
        )

    raise ValueError("Configure GOOGLE_API_KEY or OPENAI_API_KEY in settings.")

# -----
# Text Chunking
# ----- def chunk_text(text,
chunk_size=4000, overlap=400):
    """Split text into overlapping chunks for better AI processing."""
    chunks = []
    start = 0
```

```
    while start < len(text):
end = start + chunk_size
chunk = text[start:end]
chunks.append(chunk)
start = end - overlap

    return chunks

# -----
# Gemini Generator
# -----

def _generate_with_gemini(prompt: str) -> str:    """Generate raw text using
Gemini SDK."""    if not HAS_GEMINI_SDK:        raise ValueError("Gemini
SDK not installed. Run: pip install google-genai")

    if not getattr(settings, "GOOGLE_API_KEY", None):        raise
ValueError("GOOGLE_API_KEY missing in Django settings.")

    client = genai.Client(api_key=settings.GOOGLE_API_KEY)

    preferred_model = getattr(settings, "GEMINI_MODEL", "gemini-2.0-flash")
candidate_models = [    preferred_model,
    "gemini-2.0-flash",
    "gemini-flash-latest",
]

    last_err = None    for model_name in
candidate_models:        try:
```

```
        response = client.models.generate_content(
model=model_name,          contents=prompt,
        )
        if response and getattr(response, "text", None):
            return response.text
except Exception as e:
    last_err = e
continue

if last_err:
    raise last_err
return ""

# -----
# AI Response Parser
# ----- def
parse_ai_questions(raw_response):
    """Parse AI response into structured question objects."""
    questions = []

    blocks = re.split(r"---QUESTION---|---END---", raw_response)

    for block in blocks:
        block = block.strip()

        if not block:
            continue

        q = {}
```

```
for line in block.split("\n"):
line = line.strip()

if line.startswith("Question:"):
    q["question_text"] = line.replace("Question:", "").strip()

elif line.startswith("Option A:"):
    q["option_a"] = line.replace("Option A:", "").strip()

elif line.startswith("Option B:"):
    q["option_b"]
= line.replace("Option B:", "").strip()
elif
line.startswith("Option C:"):
    q["option_c"] =
line.replace("Option C:", "").strip()

elif line.startswith("Option D:"):
    q["option_d"] = line.replace("Option D:", "").strip()

elif line.startswith("Correct Answer:"):
    ans =
line.replace("Correct Answer:", "").strip().upper()

if ans in ("A", "B", "C", "D"):
q["correct_answer"] = ans

elif line.startswith("Explanation:"):
    q["explanation"]
= line.replace("Explanation:", "").strip()

required = [
"question_text",
```

```
        "option_a",
        "option_b",
        "option_c",
        "option_d",
        "correct_answer",
    ]

    if all(k in q for k in required):
        q.setdefault("explanation", "")
    questions.append(q)

    return questions

# ----- #
Main Question Generator #
-----

def generate_questions_from_text(text, num_questions=5):
    """Generate MCQs from educational text."""

    chunks = chunk_text(text)

    all_questions = []

    per_chunk = max(2, num_questions // len(chunks) + 1)

    for chunk in chunks[:5]: # limit chunks to avoid API overuse
        prompt = QUIZ_PROMPT.format(num_questions=per_chunk, content=chunk)
```

```
# Preferred: Direct Gemini SDK          if HAS_GEMINI_SDK and
getattr(settings, "GOOGLE_API_KEY", None):
    content = _generate_with_gemini(prompt)

# Fallback: LangChain
else:
    if not
HAS_LANGCHAIN:
        raise ImportError("Install langchain or enable Gemini SDK.")

    llm = get_llm()
    response = llm.invoke([HumanMessage(content=prompt)])          content
= response.content if hasattr(response, "content") else str(response)

    parsed = parse_ai_questions(content)

    all_questions.extend(parsed)

    if len(all_questions) >= num_questions:
break

# ----- #
Remove Duplicate Questions
# -----
seen = set()
unique_questions = []

for q in all_questions:
    key = q["question_text"].lower()
```

```
        if key not in seen:
seen.add(key)
unique_questions.append(q)

return unique_questions[:num_questions]
```

duplicate_filter.py

```
"""
QuizGenix - Duplicate Question Detection (Exact + Semantic)
"""
import
math
from django.conf import settings

# Semantic similarity - use sentence-transformers if available try:
from sentence_transformers import SentenceTransformer
_model = None

def _get_embedding_model():
    global _model
    if _model is None:
        _model = SentenceTransformer('all-MiniLM-L6-v2')    return _model

def get_embedding(text):
    model = _get_embedding_model()    return
model.encode(text, convert_to_numpy=True).tolist()

def cosine_similarity(a, b):
    dot = sum(x * y for x, y in zip(a, b))
    norm_a = math.sqrt(sum(x * x for x in a))
```

```
norm_b = math.sqrt(sum(x * x for x in b))    if
norm_a == 0 or norm_b == 0:
    return 0    return dot /
(norm_a * norm_b)

HAS_EMBEDDINGS = True except
ImportError:
HAS_EMBEDDINGS = False

def get_embedding(text):
    return None

def cosine_similarity(a, b):
    return 0

def exact_duplicate(question_text, existing_questions):
    """Check if question is exact duplicate."""    q_clean =
question_text.strip().lower()    for eq in
existing_questions:    if eq.strip().lower() == q_clean:
        return    True
    return False

def semantic_duplicate(question_text, existing_embeddings, threshold=None):
    """Check if question is semantically similar to existing ones."""
    if not HAS_EMBEDDINGS or not existing_embeddings:
        return False    threshold = threshold or getattr(
        settings, 'SEMANTIC_SIMILARITY_THRESHOLD', 0.85
    )
    emb = get_embedding(question_text)
    if emb is None:    return False
```

```
    for ex_emb in existing_embeddings:        if ex_emb and
cosine_similarity(emb, ex_emb) >= threshold:    return True
return False
```

```
def filter_duplicates(questions, existing_question_texts=None, existing_embeddings=None):
    """Filter out exact and semantic duplicates from question list."""
    existing_texts = list(existing_question_texts or [])    existing_embs
= list(existing_embeddings or [])    filtered = []    for q in questions:
        text = q.get('question_text', "")        if
exact_duplicate(text, existing_texts):
            continue
        emb = get_embedding(text) if HAS_EMBEDDINGS else None
if emb and semantic_duplicate(text, existing_embs):        continue
filtered.append(q)        existing_texts.append(text)        if emb:
            existing_embs.append(emb)
return filtered
```

pdf_extractor.py

```
"""
QuizGenix - PDF Text Extraction
""" import re from io
import BytesIO

try:
    import pdfplumber
    HAS_PDFPLUMBER = True except
ImportError:
    HAS_PDFPLUMBER = False
```

```
try:
    from pypdf import PdfReader
HAS_PYPDF = True except
ImportError:
    HAS_PYPDF = False

def extract_text_from_pdf(file_or_path):
    """Extract text from PDF. Returns cleaned text."""
    content = _read_pdf_content(file_or_path)    return
    _clean_text(content)

def _read_pdf_content(file_or_path):
    """Read raw text from PDF using available library."""
    if hasattr(file_or_path, 'read'):    file_obj =
    file_or_path    if hasattr(file_obj, 'seek'):
    file_obj.seek(0) else:
        file_obj = open(file_or_path, 'rb')

    text = " try:    if HAS_PDFPLUMBER:
with pdfplumber.open(file_obj) as pdf:
for page in pdf.pages:                t =
page.extract_text()                if t:
text += t + '\n'    elif HAS_PYPDF:
        reader = PdfReader(file_obj)
for page in reader.pages:                t
= page.extract_text()                if t:
text += t + '\n'    else:
```

```
        raise ImportError('Install pdfplumber or pypdf for PDF extraction.')
finally:    if not hasattr(file_or_path, 'read'):
            file_obj.close()
return text
```

```
def _clean_text(text):    """Clean
extracted text."""    if not text:
return "    text = re.sub(r'\s+', '', text)
text = re.sub(r'\n{3,}', '\n\n', text)
return text.strip()
```

personalized_quiz

```
"""
```

QuizGenix - Personalized Quiz Generation (Weak Areas)

```
"""
```

```
from django.db.models import Count
from quizzes.models import Question, PerformanceRecord, Quiz
from .ai_generator import generate_questions_from_text from
.duplicate_filter import filter_duplicates
```

```
def get_weak_concepts(user, source_text=None, quiz=None):
    """Identify concepts/topics user got wrong. Returns text snippets for AI to focus on."""
    incorrect = PerformanceRecord.objects.filter(    user=user,    is_correct=False
    ).select_related('question',    'quiz')
    if quiz:
        incorrect = incorrect.filter(quiz=quiz)
    weak_questions = [pr.question for pr in incorrect[:20]]
    if not weak_questions and source_text:    return
    source_text # No weak areas, use full content
```

```
# Build context from incorrect questions    context_parts = []    for q in weak_questions:
        context_parts.append(f"Topic/concept:        {q.question_text[:200]}...

Correct:
{q.get_option(q.correct_answer)}")
context  =  "\n\n".join(context_parts)
if source_text:
    context = f"Focus on these weak areas when generating questions:\n{context}\n\nFull
content:\n{source_text}"    return context or source_text or "Generate general practice questions."

def get_existing_question_data(quiz):
    """Get texts and embeddings for duplicate filtering."""
    questions = quiz.questions.all()    texts =
    [q.question_text for q in questions]

    embeddings = [q.embedding for q in questions if q.embedding]    return
    texts, embeddings

def create_personalized_quiz(user, source_text, title, created_by, num_questions=5):
    """Create quiz focused on user's weak areas."""
    weak_context = get_weak_concepts(user, source_text)

    raw_questions        =        generate_questions_from_text(weak_context,
num_questions=num_questions)    quiz = Quiz.objects.create(
        title=title,
        description='Personalized quiz based on your weak areas',
        created_by=created_by,        is_ai_generated=True,
        source_text=source_text[:5000],        is_personalized=True,
    )

    from .duplicate_filter import get_embedding, HAS_EMBEDDINGS
    for i, qd in enumerate(filter_duplicates(raw_questions)):
```

```
emb = get_embedding(qd['question_text']) if HAS_EMBEDDINGS else None
Question.objects.create(
    quiz=quiz,
    question_text=qd['question_text'],
    option_a=qd['option_a'],
    option_b=qd['option_b'],
    option_c=qd['option_c'],
    option_d=qd['option_d'],
    correct_answer=qd['correct_answer'],
    explanation=qd.get('explanation', ''), order=i,
    embedding=emb,
) return
quiz
```

student_creator.py

```
"""
```

QuizGenix - Student Creation from Excel

```
""" import re from django.contrib.auth import
get_user_model from openpyxl import
load_workbook from io import BytesIO
```

```
User = get_user_model()
```

```
def generate_username(name, email):
```

```
    """Generate username from name and email.""" base =
    re.sub(r'^a-zA-Z0-9', '', name.lower())[:10] email_part =
    re.sub(r'^a-zA-Z0-9', '', email.split('@')[0][:5]) return
    f'{base}_{email_part}'.lower()[:30]
```

```
def generate_password(name, email):
    """Generate password from name and email."""
import hashlib

    raw = f'{name}{email}'.encode()
    return
    hashlib.sha256(raw).hexdigest()[:12]

def create_students_from_excel(file, teacher):
    """Process Excel file and create student accounts. Returns list of (student, username,
password)."""
    wb = load_workbook(filename=BytesIO(file.read()), read_only=True, data_only=True)
ws = wb.active

    results = []

    for row in ws.iter_rows(min_row=2, values_only=True):
        if not row or len(row) < 2:
            continue

        name = str(row[0]).strip() if row[0] else "
email = str(row[1]).strip() if row[1] else "
        if
        not name or not email or '@' not in email:
            continue
        if
        User.objects.filter(email=email).exists():
            continue

        username = generate_username(name, email)
        base_username = username

        c = 1
        while
        User.objects.filter(username=username).exists():
            username = f'{base_username}{c}'

            c += 1

        password = generate_password(name, email)
        user = User.objects.create_user(
```

```
username=username,          email=email,
password=password,
first_name=name.split()[0] if name else "",
    last_name=' '.join(name.split()[1:]) if len(name.split()) > 1 else "",
role=User.Role.STUDENT,      is_approved=True,
created_by=teacher,
    )

    from quizzes.models import StudentCredential
StudentCredential.objects.create(
student=user,
generated_username=username,
generated_password=password,
created_by=teacher,
    )

    results.append((user, username, password))
return results
```

11.2 List of Abbreviations and Symbols Used

Abbreviation Description

AI	Artificial Intelligence
NLP	Natural Language Processing
LLM	Large Language Model
MCQ	Multiple Choice Question
API	Application Programming Interface
PDF	Portable Document Format
ORM	Object Relational Mapper
EDA	Exploratory Data Analysis

11.3 Journal Certification



CERTIFICATE OF PUBLICATION

ISSN: 2583-6129 || IMPACT FACTOR: 8.072

INTERNATIONAL SCIENTIFIC JOURNAL OF ENGINEERING AND MANAGEMENT

An International Scholarly || Multidisciplinary || Open Access || Peer-Reviewed Journal

This is to certify that the article authored by

A. Sathvika

entitled

A Framework for Automated Multiple-Choice Question Generation and Evaluation using

Langchain and Gemini

has been successfully peer-reviewed and published in the Conference Proceedings of the International Scientific Journal of Engineering and Management (ISJEM) as part of the **International Conference on Intelligent Multidisciplinary Engineering System – ACME Tech@2K26**.

The article has been published in **Volume 05, ACME 2K26** Conference Proceedings, March 2026 and has been assigned the following Digital Object Identifier **doi : 10.55041/ISJEM.ACME082**



Editor-in-Chief

International Scientific Journal of Engineering and Management

#62/1, New No. 7, 1st Cross, 2nd Main, Ganganagar, R.T. Nagar, Bangalore North, Bangalore, Karnataka – 560032, India

editor@isjem.com ||  www.isjem.com



CERTIFICATE OF PUBLICATION

ISSN: 2583-6129 || IMPACT FACTOR: 8.072



INTERNATIONAL SCIENTIFIC JOURNAL OF ENGINEERING AND MANAGEMENT

An International Scholarly || Multidisciplinary || Open Access || Peer-Reviewed Journal

This is to certify that the article authored by

T. Dileep

entitled

A Framework for Automated Multiple-Choice Question Generation and Evaluation using Langchain and Gemini

has been successfully peer-reviewed and published in the Conference Proceedings of the
International Scientific Journal of Engineering and Management (ISJEM) as part of the

International Conference on Intelligent Multidisciplinary Engineering System – ACME Tech@2K26.

The article has been published in **Volume 05, ACME 2K26** Conference Proceedings, March 2026 and has been assigned the
following Digital Object Identifier **doi : 10.55041/ISJEM.ACME082**



Editor-in-Chief

International Scientific Journal of Engineering and Management

#62/1, New No. 7, 1st Cross, 2nd Main, Gangaganagar, R.T. Nagar, Bangalore North, Bangalore, Karnataka – 560032, India

editor@isjem.com || www.isjem.com





CERTIFICATE OF PUBLICATION

ISSN: 2583-6129 || IMPACT FACTOR: 8.072



INTERNATIONAL SCIENTIFIC JOURNAL OF ENGINEERING AND MANAGEMENT

An International Scholarly || Multidisciplinary || Open Access || Peer-Reviewed Journal

This is to certify that the article authored by

D. Lakshitha

entitled

A Framework for Automated Multiple-Choice Question Generation and Evaluation using Langchain and Gemini

has been successfully peer-reviewed and published in the Conference Proceedings of the
International Scientific Journal of Engineering and Management (ISJEM) as part of the

International Conference on Intelligent Multidisciplinary Engineering System – ACME Tech@2K26.

The article has been published in **Volume 05, ACME 2K26** Conference Proceedings, March 2026 and has been assigned the
following Digital Object Identifier **doi : 10.55041/ISJEM.ACME082**



Editor-in-Chief

International Scientific Journal of Engineering and Management

#62/1, New No. 7, 1st Cross, 2nd Main, Ganganagar, R.T. Nagar, Bangalore North, Bangalore, Karnataka – 560032, India

editor@isjem.com || www.isjem.com





CERTIFICATE OF PUBLICATION

ISSN: 2583-6129 || IMPACT FACTOR: 8.072



INTERNATIONAL SCIENTIFIC JOURNAL OF ENGINEERING AND MANAGEMENT

An International Scholarly || Multidisciplinary || Open Access || Peer-Reviewed Journal

This is to certify that the article authored by

B. Akhila

entitled

A Framework for Automated Multiple-Choice Question Generation and Evaluation using Langchain and Gemini

has been successfully peer-reviewed and published in the Conference Proceedings of the
International Scientific Journal of Engineering and Management (ISJEM) as part of the

International Conference on Intelligent Multidisciplinary Engineering System – ACME Tech@2K26.

The article has been published in **Volume 05, ACME 2K26** Conference Proceedings, March 2026 and has been assigned the
following Digital Object Identifier **doi : 10.55041/ISJEM.ACME082**



Editor-in-Chief

International Scientific Journal of Engineering and Management

#62/1, New No. 7, 1st Cross, 2nd Main, Ganganagar, R.T. Nagar, Bangalore North, Bangalore, Karnataka – 560032, India

editor@isjem.com || www.isjem.com





CERTIFICATE OF PUBLICATION

ISSN: 2583-6129 || IMPACT FACTOR: 8.072



INTERNATIONAL SCIENTIFIC JOURNAL OF ENGINEERING AND MANAGEMENT

An International Scholarly || Multidisciplinary || Open Access || Peer-Reviewed Journal

This is to certify that the article authored by

L. Gayathri

entitled

A Framework for Automated Multiple-Choice Question Generation and Evaluation using Langchain and Gemini

has been successfully peer-reviewed and published in the Conference Proceedings of the
International Scientific Journal of Engineering and Management (ISJEM) as part of the

International Conference on Intelligent Multidisciplinary Engineering System – ACME Tech@2K26.

The article has been published in **Volume 05, ACME 2K26** Conference Proceedings, March 2026 and has been assigned the
following Digital Object Identifier **doi : 10.55041/ISJEM.ACME082**



Editor-in-Chief

International Scientific Journal of Engineering and Management

#62/1, New No. 7, 1st Cross, 2nd Main, Ganganagar, R.T. Nagar, Bangalore North, Bangalore, Karnataka – 560032, India

editor@isjem.com || www.isjem.com



ANNAMACHARYA INSTITUTE OF TECHNOLOGY AND SCIENCES ::TIRUPATI



(Autonomous)

Approved by AICTE, New Delhi & Affiliation to JNTUA, Anantapuramu.

All the 5 eligible UG Engineering Programs are accredited by NBA, New Delhi

Accredited by NAAC with 'A' Grade, Bangalore. Accredited by Institution of Engineers (India), KOLKATA.

Recognized under sections 2(f) & 12(B) of UGC Act 1956.



International Conference on Intelligent Multidisciplinary Engineering System

ACME TECH - 2K26

28, March 2026



This certificate is awarded to **Ms. Ande Sathvika**, for having presented a paper with
of **Annamacharya institute of technology and science college**,
title **“ AutoAssessAI: An Intelligent Framework for Automated Quiz Generation and Adaptive Evaluation using Large Language Models ”**

in the International Conference on Intelligent Multidisciplinary Engineering System (ACMETECH-2K26)
organized by **ANNAMACHARYA INSTITUTE OF TECHNOLOGY AND SCIENCES ::TIRUPATI**, Andhra
Pradesh, India, on 28th March 2K26.

Dr. Shaik Jumlesha
Organizing Chair

ISBN : 978-93-5773-360-1

Dr. C. Nadhamuni Reddy
Conference Chair & Principal

ANNAMACHARYA INSTITUTE OF TECHNOLOGY AND SCIENCES ::TIRUPATI



(Autonomous)

Approved by AICTE, New Delhi & Affiliation to JNTUA, Anantapuramu.

All the 5 eligible UG Engineering Programs are accredited by NBA, New Delhi

Accredited by NAAC with 'A' Grade, Bangalore. Accredited by Institution of Engineers (India), KOLKATA.



International Conference on Intelligent Multidisciplinary Engineering System

ACME TECH - 2K26

28, March 2026



This certificate is awarded to **Mr. T Dileep**
of **Annamacharya institute of technology and sciences, tirupati**, for having presented a paper with
title **" AutoAssessAI: An Intelligent Framework for Automated Quiz Generation and Adaptive Evaluation using
Large Language Models "**

in the International Conference on Intelligent Multidisciplinary Engineering System (ACMETECH-2K26)
organized by **ANNAMACHARYA INSTITUTE OF TECHNOLOGY AND SCIENCES ::TIRUPATI**, Andhra
Pradesh, India, on 28th March 2K26.

ISBN : 978-93-5773-360-1

Dr.Shaik Jumlesha
Organizing Chair

Dr.C.Nadhamuni Reddy
Conference Chair & Principal

ANNAMACHARYA INSTITUTE OF TECHNOLOGY AND SCIENCES ::TIRUPATI



(Autonomous)

Approved by AICTE, New Delhi & Affiliation to JNTUA, Anantapuramu.

All the 5 eligible UG Engineering Programs are accredited by NBA, New Delhi

Accredited by NAAC with 'A' Grade, Bangalore. Accredited by Institution of Engineers (India), KOLKATA.

Recognized under sections 2(f) & 12(B) of UGC Act 1956.



International Conference on Intelligent Multidisciplinary Engineering System

ACME TECH - 2K26

28, March 2026



This certificate is awarded to Ms. Dumpa Lakshitha
of Annamacharya Institute of Technology and Sciences Tirupati, for having presented a paper with
title "AutoAssessAI: An Intelligent Framework for Automated Quiz Generation and Adaptive Evaluation using
Large Language Models”

in the International Conference on Intelligent Multidisciplinary Engineering System (ACMETECH-2K26)
organized by **ANNAMACHARYA INSTITUTE OF TECHNOLOGY AND SCIENCES ::TIRUPATI**, Andhra
Pradesh, India, on 28th March 2K26.

Dr.Shaik Jumlesha
Organizing Chair

ISBN : 978-93-5773-360-1

Dr.C.Nadhamuni Reddy
Conference Chair & Principal

ANNAMACHARYA INSTITUTE OF TECHNOLOGY AND SCIENCES ::TIRUPATI



(Autonomous)

Approved by AICTE, New Delhi & Affiliation to JNTUA, Anantapuramu.

All the 5 eligible UG Engineering Programs are accredited by NBA, New Delhi

Accredited by NAAC with 'A' Grade, Bangalore. Accredited by Institution of Engineers (India), KOLKATA.

Recognized under sections 2(f) & 12(B) of UGC Act 1956.



International Conference on Intelligent Multidisciplinary Engineering System

ACME TECH - 2K26

28, March 2026



This certificate is awarded to **Ms. Akhila Bandapalli** of **Annamacharya institute of technology and sciences , Tirupati** , for having presented a paper with title **" AutoAssessAI: An Intelligent Framework for Automated Quiz Generation and Adaptive Evaluation using Large Language Models "**

in the International Conference on Intelligent Multidisciplinary Engineering System (ACMETECH-2K26) organized by **ANNAMACHARYA INSTITUTE OF TECHNOLOGY AND SCIENCES ::TIRUPATI** , Andhra Pradesh, India, on 28th March 2K26.

ISBN : 978-93-5773-360-1

Dr.Shaik Jumlesha
Organizing Chair

Dr.C.Nadhamuni Reddy
Conference Chair & Principal

ANNAMACHARYA INSTITUTE OF TECHNOLOGY AND SCIENCES ::TIRUPATI



(Autonomous)

Approved by AICTE, New Delhi & Affiliation to JNTUA, Anantapuramu.

All the 5 eligible UG Engineering Programs are accredited by NBA, New Delhi

Accredited by NAAC with 'A' Grade, Bangalore. Accredited by Institution of Engineers (India), KOLKATA.



Recognized under sections 2(f) & 12(B) of UGC Act 1956.

International Conference on Intelligent Multidisciplinary Engineering System

ACME TECH - 2K26

28, March 2026



This certificate is awarded to **Ms. Gayathri L**
of **Annamacharya institute of technology and sciences tirupati**, for having presented a paper with
title “ **AutoAssessAI: An Intelligent Framework for Automated Quiz Generation and Adaptive Evaluation**
using Large Language Models

in the International Conference on Intelligent Multidisciplinary Engineering System (ACMETECH-2K26)
organized by **ANNAMACHARYA INSTITUTE OF TECHNOLOGY AND SCIENCES ::TIRUPATI**, Andhra
Pradesh, India, on 28th March 2K26.

ISBN : 978-93-5773-360-1

Dr.Shaik Jumlesha
Organizing Chair

Dr.C.Nadhamuni Reddy
Conference Chair & Principal



A Framework for Automated Multiple-Choice Question Generation and Evaluation using Langchain and Gemini

M. A. Sathvika¹

Asst. Professor, Department of
 AIDS
 Annamacharya Institute of
 Technology and Sciences, Tirupati
 – 517520, A.P.
Andesathvika2602@gmail.com

T. Dileep²

Student Department of AIDS
 Annamacharya Institute of
 Technology and Sciences, Tirupati
 – 517520, A.P.
dileepthummala4@gmail.com

D. Lakshitha³

Student Department of AIDS
 Annamacharya Institute of
 Technology and Sciences, Tirupati
 – 517520, A.P.
lakshithadumpa@gmail.com

B. Akhila⁴

Student Department of AIDS
 Annamacharya Institute of
 Technology and Sciences, Tirupati
 – 517520, A.P.
akhilabandapalli@gmail.com

L. Gayathri⁵

Student Department of AIDS
 Annamacharya Institute of
 Technology and Sciences, Tirupati
 – 517520, A.P.
gayathrilakkoju771@gmail.com

Abstract:

The rapid growth of digital education platforms has created a strong demand for intelligent systems that can automatically generate and evaluate assessment content. This project presents an AI-driven solution for the automated generation and evaluation of Multiple-Choice Questions (MCQs) using LangChain and Gemini Large Language Models (LLMs). The system aims to reduce manual effort in quiz creation while ensuring high-quality, contextually accurate questions across multiple domains. The proposed system leverages LangChain as an orchestration framework to connect prompts, documents, and language models effectively. Gemini LLM is used to generate MCQs, including questions, options, correct answers, and explanations. The system also incorporates evaluation mechanisms to assess the quality of generated quizzes based on difficulty, relevance, and correctness. This ensures that the quizzes are suitable for academic and training purposes. Furthermore, the system supports dynamic content generation from user-provided input

such as text documents, PDFs, or topics. It enhances scalability in e-learning environments by enabling automatic quiz creation in real-time.

Keywords: Lang Chain, Gemini LLM, MCQ Generation, Natural Language Processing, AI in Education, Automated Assessment, Prompt Engineering

1 INTRODUCTION

The rapid advancement of digital technologies has significantly transformed the education sector, enabling new methods of teaching, learning, and assessment. One of the most important aspects of education is the evaluation of knowledge, which is commonly performed through quizzes and examinations. However, designing high-quality multiple-choice questions (MCQs) manually requires considerable time, effort, and [3] subject expertise. This challenge becomes more prominent in large-scale online learning platforms where continuous assessment is

required. As a result, there is a growing need for intelligent systems that can automate the process of quiz generation and evaluation.

In recent years, Artificial Intelligence (AI) and Natural Language Processing (NLP) have emerged as powerful tools for automating content creation. Large Language Models (LLMs), such as Gemini, are capable of understanding context, generating human-like text, and performing complex language-based [6] tasks. These models can analyze educational content and transform it into structured formats like questions and answers. This capability opens up new opportunities for automating assessment systems, reducing dependency on manual question creation while maintaining quality and relevance.

The proposed system focuses on both the generation and evaluation of multiple-choice quizzes. It accepts input in various forms, such as text documents or specific topics, and processes it to generate meaningful questions along with options, correct answers, and explanations. Additionally, the system evaluates [17] [4] the generated content based on parameters such as difficulty level, clarity, and relevance. This dual functionality ensures that the quizzes are not only automatically created but also meet educational standards, making them suitable for real-world applications.

Overall, the integration of LangChain and Gemini LLM provides a smart and efficient solution for automated assessment in modern education systems. The system reduces the workload of educators, improves consistency in question quality, and supports scalable [9] learning environments. As educational platforms continue to evolve, such AI-driven solutions will play a vital role in enhancing personalized learning and continuous evaluation, ultimately contributing to more effective and engaging educational experiences.

II. METHODOLOGY

The methodology of the proposed system is designed to automate the generation and evaluation of multiple-choice questions using advanced language models and structured processing techniques. The system follows a pipeline-based architecture where user input, such as text, documents, or specific topics, is first collected and preprocessed. This preprocessing step involves cleaning the text, removing irrelevant information, and organizing the content into a [2] format suitable for analysis. The refined input ensures that the language model receives high-quality data, which directly impacts the accuracy and relevance of the generated questions.



Figure 1: AI Engine Process

Once the input data is prepared, LangChain is used to construct a prompt-driven workflow that interacts with the Gemini Large Language Model. Prompt templates are carefully designed to instruct the model to generate [8],[14] multiple-choice questions, including the question statement, answer options, correct answer, and explanation. LangChain manages this interaction by chaining multiple components such as prompt templates, input variables, and output parsers. This structured approach ensures consistency in the generated output and allows the system to handle complex queries efficiently.

After the MCQs are generated, the system proceeds to the evaluation phase, where the quality of the questions is assessed. This

involves checking for grammatical correctness, contextual relevance, and logical consistency of the options. Additionally, [1] difficulty levels are estimated based on the complexity of the question and vocabulary used. The evaluation process may combine rule-based validation with AI-driven scoring techniques to ensure that the generated content meets educational standards. This step is essential to filter out ambiguous or incorrect questions before presenting them to the user.

Finally, the validated quiz content is presented through an interactive user interface, where users can attempt the questions and receive immediate feedback. The system also stores generated quizzes and user responses for future analysis and improvement. Performance metrics such as accuracy, [5] response time, and user scores are tracked to evaluate the effectiveness of the system. This end-to-end methodology ensures a seamless flow from input processing to quiz generation, evaluation, and user interaction, making the system efficient, scalable, and suitable for modern e-learning environments.

III. LITERATURE REVIEW

Recent advancements in Natural Language Processing have significantly contributed to automated question generation systems. Early transformer-based models such as T5 and BERT laid the foundation for generating structured educational content. These models enabled the [11] transformation of text into question-answer formats, improving the efficiency of assessment systems. However, earlier approaches often required complex pipelines for keyword extraction and rule-based processing, limiting their scalability and adaptability in dynamic learning environments.

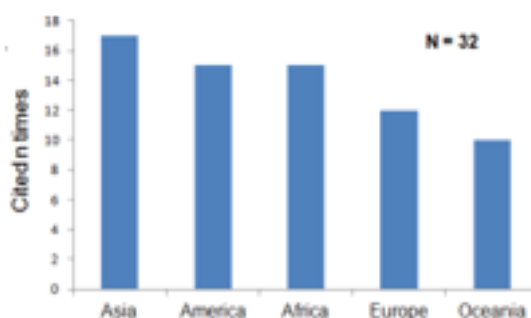


Figure 2: Performance metrics

Mucciaccia et al. (2025) proposed an advanced system that combines large language models with prompt engineering techniques to automate both MCQ generation and evaluation. Their approach integrates a [2] review mechanism to ensure the correctness and relevance of generated questions. The study demonstrates that LLM-based systems can effectively produce reliable and scalable assessment content, reducing the dependency on human experts.

Overall, the literature indicates a clear shift from traditional NLP techniques to advanced LLM-based systems for automated quiz generation and evaluation. Modern approaches emphasize prompt engineering, iterative refinement, and validation mechanisms to ensure quality and reliability. These advancements form the foundation for the proposed system, which integrates LangChain and Gemini LLM to deliver efficient and scalable MCQ generation.

IV. RELATED WORKS

Automated question generation has been widely explored in recent years as part of intelligent educational systems. Earlier works primarily relied on rule-based and template-driven approaches, where predefined grammatical rules and syntactic structures were used to generate questions from input text. These systems often included steps such as [6] keyword extraction, sentence parsing, and distractor generation. Although they provided a structured approach, their performance was limited in handling complex contexts and generating diverse questions.

With the advancement of machine learning, researchers began incorporating supervised learning techniques and transformer-based models to improve question generation quality. Models such as T5 and BERT enabled systems to [4] generate more natural and context-aware questions by learning from large datasets. These approaches significantly improved fluency and coherence but still required extensive training data and struggled with maintaining topic relevance in long documents.

Recent works have shifted towards the use of Large Language Models (LLMs), which can generate high-quality questions with minimal task-specific training. For instance, scalable frameworks have been proposed that mimic human-like question generation by combining neural models with quality control mechanisms. These systems are capable of producing large volumes of diverse question-answer pairs while filtering out low-quality outputs, thus enhancing reliability and scalability.

More advanced research focuses on improving contextual accuracy and domain adaptability. Techniques such as Retrieval-Augmented Generation (RAG) and In-Context Learning (ICL) have been introduced to ensure that generated questions [8] remain closely aligned with the source content. Hybrid models combining retrieval and generation have shown superior performance in producing contextually relevant and pedagogically meaningful questions compared to traditional methods.

Flow of the Application

The application begins by accepting input from the user in the form of text, documents, or specific topics. This input is first passed through a preprocessing stage where unnecessary characters, [7] formatting issues, and irrelevant data are removed to ensure clarity and consistency. Once cleaned, the processed content is fed into a LangChain pipeline, which structures the input using predefined prompt templates.

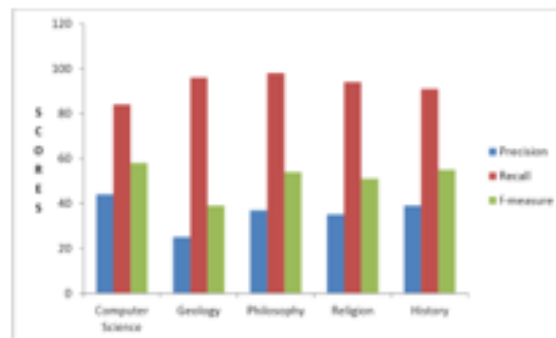


Figure 3: Producing Contextually

These prompts guide the Gemini Large Language Model to generate multiple-choice questions, including the question statement, answer options, correct answer, and explanations. The generation process is dynamic and [14] adapts based on the complexity and context of the input provided by the user.

After the questions are generated, the system performs an evaluation phase to ensure quality and accuracy. This includes checking grammatical correctness, relevance to the original content, and logical consistency of answer options. The validated quiz is then displayed through an interactive user interface where users can attempt the questions and receive immediate feedback. Additionally, user responses and performance metrics are stored for analysis, enabling continuous improvement of the system.

Articles with question generation

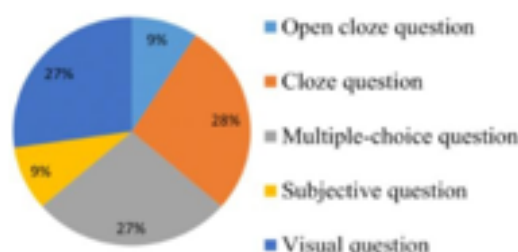


Figure 4: Quiz Generation

This structured flow ensures seamless interaction from input submission to quiz generation, evaluation, and result presentation,

making the application efficient and user-friendly.

Dataset Collection

The dataset for this project is gathered from a variety of reliable educational sources to ensure diversity and quality of content. These sources include digital textbooks, academic articles, open educational resources, and publicly available datasets related to different subjects such as science, technology, and [5] general knowledge. The goal of collecting data from multiple domains is to enable the system to generate a wide range of multiple-choice questions that are contextually rich and suitable for various learning levels.

Once the data is collected, it undergoes a preprocessing phase to improve its usability. This includes removing noise such as special characters, irrelevant symbols, and duplicate content, as well as segmenting the text into meaningful units like paragraphs or sentences. The cleaned dataset is then organized into structured formats that can be easily processed by the [10] [3] LangChain framework. Proper formatting ensures that the input provided to the language model is clear, which directly enhances the quality and relevance of the generated questions.

In addition to raw textual data, benchmark datasets and sample question banks are also incorporated to validate the performance of the system. These datasets help in evaluating the accuracy, difficulty level, and correctness of the generated MCQs. By combining diverse [3] data sources with structured preprocessing and validation techniques, the dataset collection process ensures that the system is capable of producing high-quality, reliable, and educationally meaningful quiz content.

Algorithm

The algorithm for the proposed system follows a structured sequence of steps to generate and evaluate multiple-choice questions efficiently.

Initially, the system accepts input in the form of text, documents, or user-defined topics. This input is preprocessed to remove noise, normalize the content, and divide it [16] into meaningful segments. After preprocessing, a prompt template is created using LangChain, which defines the structure and requirements for generating MCQs. The processed input, along with the prompt, is then passed to the Gemini Large Language Model, which generates questions, multiple answer options, the correct answer, and detailed explanations in a structured format.

Following the generation phase, the algorithm includes a validation and evaluation stage to ensure the quality of the output. Each generated question is checked for grammatical correctness, contextual relevance, and logical consistency among the answer options. The system may also assign a difficulty level based on factors such as [2],[11] vocabulary complexity and conceptual depth. Once validated, the final set of MCQs is displayed to the user through the interface, where responses can be recorded and evaluated.

V DISCUSSION AND RESULTS

The implementation of an automated MCQ generation system using LangChain and Gemini LLM demonstrates significant improvements in efficiency and scalability compared to traditional manual methods. The system is capable of generating a large number of questions within a short time, reducing the workload on educators and content creators. The use of advanced language models ensures that the generated questions are contextually relevant and linguistically accurate.

Performance evaluation shows that the system significantly reduces the time required for quiz creation compared to manual methods. Multiple questions can be generated within seconds, making it suitable for real-time applications in e-learning platforms.

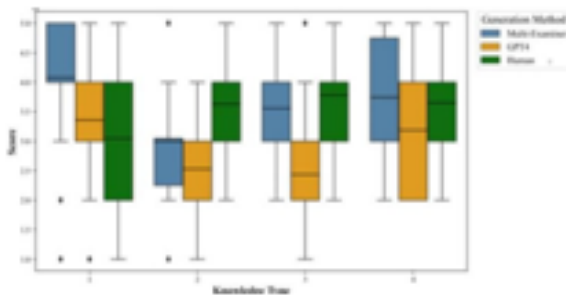


Figure 5: Human Intervention.

The evaluation module successfully filters out most irrelevant or low-quality questions, ensuring that only meaningful and usable content is presented to users. This contributes to maintaining a reliable standard of assessment without requiring extensive human intervention.

User interaction results indicate that the system provides a smooth and intuitive experience. The interface allows users to easily input content, generate quizzes, and receive immediate feedback on their responses. The inclusion of explanations for each question enhances [2] understanding and supports self-learning. Additionally, performance tracking features such as scoring and accuracy analysis help users monitor their progress and identify areas for improvement.

Despite these positive outcomes, some challenges were observed during testing. In certain cases, the system produced questions with slight ambiguity or uneven difficulty levels, especially when the input content was highly complex or unstructured. However, these issues were relatively minimal and can be addressed through improved prompt design and further refinement of evaluation techniques. Overall, the results indicate that the system is effective, efficient, and suitable for practical use in automated educational assessment.

CONTRIBUTION

This project contributes to the field of AI-driven education by developing an automated system for generating and evaluating multiple-choice questions using advanced language models. It introduces a structured approach that combines

prompt engineering with a pipeline-based framework to produce high-quality quiz content from various input sources. By integrating LangChain with the Gemini Large Language Model, the system ensures efficient handling of [2] input data, consistent output generation, and improved contextual understanding. This contribution helps reduce manual effort in quiz creation while maintaining accuracy and relevance.

In addition, the project provides a comprehensive solution that not only generates questions but also evaluates their quality using automated validation techniques. The inclusion of explanations and performance tracking enhances the learning experience and supports continuous assessment. The system's ability to adapt to different domains and input formats makes it versatile for educational institutions, training platforms, and self-learning applications. Overall, this work contributes a scalable and intelligent framework that advances the use of artificial intelligence in modern assessment systems.

RELEVANCE

The proposed system is highly relevant in the current educational landscape, where digital learning and online assessments are becoming increasingly important. With the rapid growth of e-learning platforms, there is a strong demand for efficient and scalable methods to create assessment content. Automated quiz generation using advanced AI technologies [1] addresses this need by enabling quick and consistent production of multiple-choice questions. This is particularly beneficial for educators and institutions that require frequent evaluations without investing excessive time and effort in manual question preparation.

VII. CONCLUSION

This project presents an effective approach for automating the generation and evaluation of multiple-choice questions using LangChain and the Gemini Large Language Model. The system

successfully demonstrates how advanced natural language processing techniques can be applied to create [6] meaningful and contextually accurate quiz content from various input sources. By reducing the need for manual intervention, the proposed solution improves efficiency and consistency in assessment creation, making it highly suitable for modern educational environments.

In conclusion, the proposed system highlights the potential of combining artificial intelligence with educational tools to create scalable and intelligent assessment solutions. While there are minor limitations related to input quality and evaluation precision, the overall performance of the system is promising. With further enhancements, such as adaptive learning capabilities and multilingual support, the system can be extended to meet a wider range of educational needs and play a significant role in the future of AI-driven learning.

VIII. FUTURE WORK

Future enhancements of the proposed system can focus on improving the adaptability and intelligence of quiz generation. One important direction is the development of adaptive learning capabilities, where the system adjusts the difficulty level of questions based on the user's performance. This would create a more personalized learning experience, allowing users to progress at their own pace while focusing on areas that require improvement. Incorporating advanced feedback mechanisms can further enhance the effectiveness of the learning process.

REFERENCES

- [1]. Brown, T. B., et al., "Language Models are Few-Shot Learners," *Advances in Neural Information Processing Systems (NeurIPS)*, 2020.
- [2]. Raffel, C., et al., "Exploring the Limits of Transfer Learning with a Unified Text-to-

Text Transformer," *Journal of Machine Learning Research*, 2020.

- [3]. Lewis, P., et al., "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks," *NeurIPS*, 2020.

- [4]. Devlin, J., et al., "BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding," *NAACL*, 2021.

- [5]. Radford, A., et al., "Learning Transferable Visual Models from Natural Language Supervision," *ICML*, 2021.

- [6]. OpenAI, "Advancements in Large Language Models for Educational Applications," *Technical Report*, 2023.

11.5 References/Bibliography

- [1]. Brown, T. B., et al., “Language Models are Few-Shot Learners,” Advances in Neural Information Processing Systems (NeurIPS), 2020.
- [2]. Raffel, C., et al., “Exploring the Limits of Transfer Learning with a Unified Text-to-Text Transformer,” Journal of Machine Learning Research, 2020.
- [3]. Lewis, P., et al., “Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks,” NeurIPS, 2020.
- [4]. Devlin, J., et al., “BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding,” NAACL, 2021.
- [5]. Radford, A., et al., “Learning Transferable Visual Models from Natural Language Supervision,” ICML, 2021.
- [6]. OpenAI, “Advancements in Large Language Models for Educational Applications,” Technical Report, 2023.